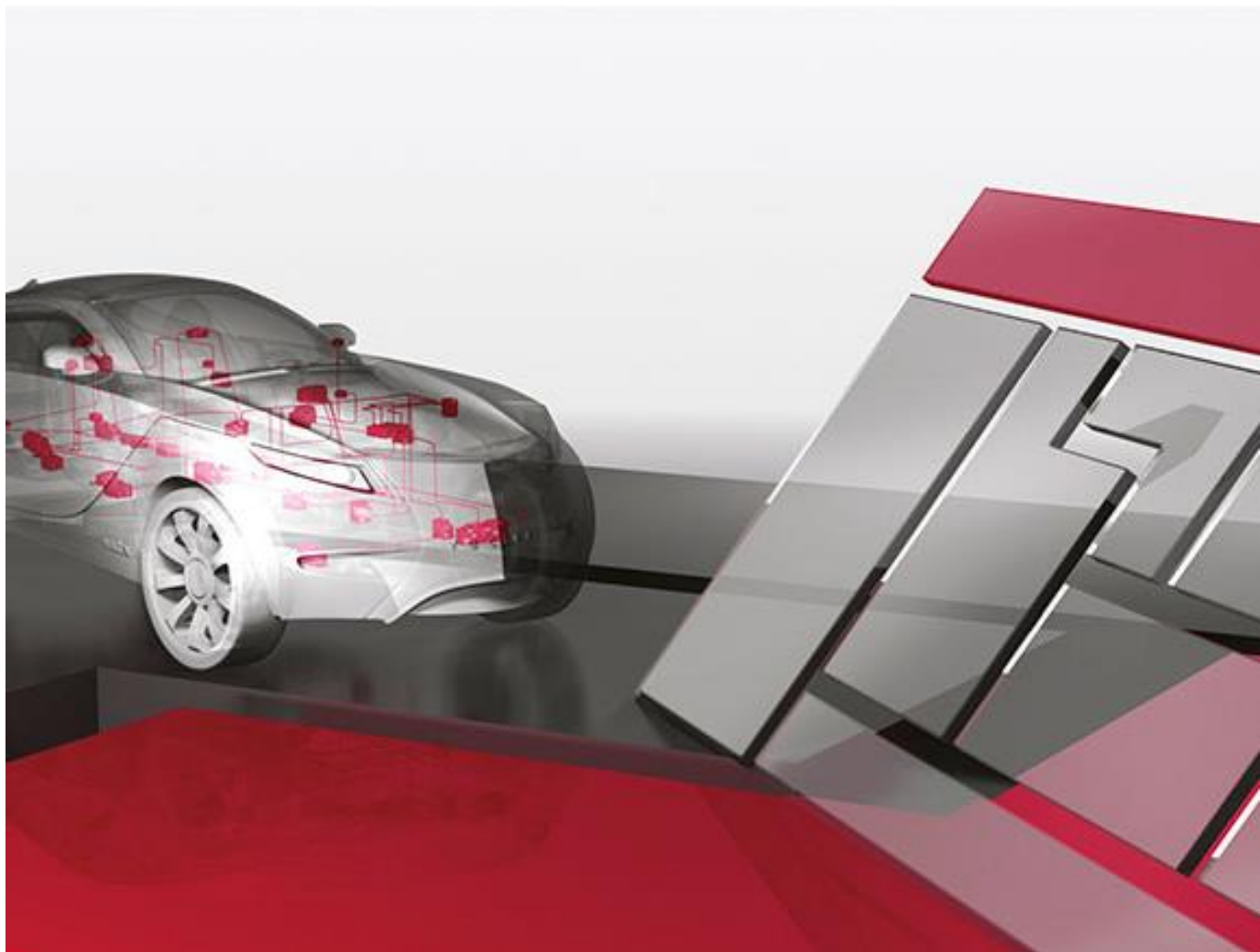




UserManual

Start Application

Start Application

Version 1.14
English

Imprint

Vector Informatik GmbH
Ingersheimer Straße 24
D-70499 Stuttgart

Vector reserves the right to modify any information and/or data in this user documentation without notice. Neither this documentation nor any of its parts may be reproduced in any form or by any means without the prior written consent of Vector. To the maximum extent permitted under law, all technical data, texts, graphics, images and their design are protected by copyright law, various international treaties and other applicable law. Any unauthorized use may violate copyright and other applicable laws or regulations.

© Copyright 2015, Vector Informatik GmbH. Printed in Germany.
All rights reserved.

Contents

1	Introduction	3
1.1	About This User Manual	3
1.1.1	Certification	4
1.1.2	Warranty	4
1.1.3	Support	4
1.1.4	Trademarks	4
1.1.5	Example Code	5
1.2	Overview	5
2	Bus Independent Functionality – High Level	7
2.1	Basic Architecture	7
2.2	CANoe Version Dependencies	8
2.3	Getting Started	10
2.4	Use Case COM	12
2.4.1	SWC	12
2.4.2	CANoe	13
2.5	Use Case COM – J1939RM Extension	14
2.5.1	SWC	14
2.5.2	CANoe	15
2.6	Use Case COM – SecOC Extension	16
2.6.1	SWC	16
2.6.2	CANoe	17
2.7	Use Case DIAG	18
2.7.1	SWC	18
2.7.2	CFG5	19
2.7.3	CANoe	21
2.8	Use Case ETH	23
2.8.1	SWC	23
2.8.2	CANoe	24
2.9	Use Case FBL	25
2.9.1	SWC	25
2.9.2	CANoe	26
2.10	Use Case J1939DIAG	27
2.10.1	SWC	27
2.10.2	CFG5	28
2.10.3	CANoe	29
2.11	Use Case MEM	30
2.11.1	SWC	30
2.11.2	CFG5	31
2.11.3	CANoe	32
2.12	Use Case MemoryProtection	34
2.12.1	SWC	34
2.12.2	CFG5	35
2.12.3	CANoe	37
2.13	Use Case Multicore	38
2.13.1	SWC	38
2.13.2	CFG5	39
2.13.3	CANoe	40

2.14	Use Case NM	41
2.14.1	SWC	41
2.14.2	CFG5	42
2.14.3	CANoe	43
2.15	Use Case SCC	44
2.15.1	Software Design	44
2.15.2	CANoe	45
2.16	Use Case WDG	46
2.16.1	SWC	46
2.16.2	CFG5	47
2.16.3	CANoe	47
2.17	Use Case XCP	48
2.17.1	SWC	48
2.17.2	CANoe	49
2.18	Multiplexing of Variables	50
2.19	Using Start Application in an IDM Configuration	53
2.19.1	SWC	53
2.19.2	CFG5	53
2.19.3	Code	54
2.19.4	CANoe	54
2.20	Using Start Application without RTE	55
3	Bus Specific Functionality – Low Level	56
3.1	ECU Data Mapping	56
3.2	CANoe Data Mapping	57
3.3	Bus Specific Mapping in this Delivery	58
4	Additional Information	59
4.1	Related Files	59
4.2	VTT Dual Target	62
4.3	Code Template Generation	63
4.4	Adapt MakeSupport	63
4.4.1	Useful Commands	64
4.4.2	Change Compiler Settings	65
4.4.3	Add and Delete *.c and *.h file (e.g. AR Software Components)	65
4.4.4	Change the Project Name	66
4.4.5	Deactivate MICROSAR BSW Modules	66
4.4.6	Advanced MakeSupport Settings	66
5	Appendix A: Addresses	67

1 Introduction

1.1 About This User Manual

To Find information quickly

This user manual provides you with the following access help:

- > At the beginning of each chapter you will find a summary of the contents
- > The header shows in which chapter of the manual you are
- > The footer shows the version of the manual
- > At the end of the user manual you will find a glossary to look-up used technical terms
- > At the end of the user manual an index will help you to find information quickly

Conventions

In the two tables below, you will find the notation and icon conventions used throughout the manual.

Style	Utilization
bold	Fields/blocks, user/surface interface elements, window- and dialog names of the software, special emphasis of terms. [OK] Push buttons in square brackets File Save Notation for menus and menu entries
MICROSAR	Legally protected proper names.
Source Code	File and directory names, source code, class and object names, object attributes and values
Hyperlink	Hyperlinks and references.
<CTRL>+<S>	Notation for shortcuts.

Symbol	Utilization
	This icon indicates notes and tips that facilitate your work.
	This icon warns of dangers that could lead to damage.
	This icon indicates more detailed information.
	This icon indicates examples.
	This icon indicates step-by-step instructions.
	This icon indicates text areas where changes of the currently described file are allowed or necessary.

Symbol	Utilization
	This icon indicates files you must not change.
	This icon indicates multimedia files like e.g. video clips.
	This icon indicates an introduction into a specific topic.
	This icon indicates text areas containing basic knowledge.
	This icon indicates text areas containing expert knowledge.
	This icon indicates that something has changed.

1.1.1 Certification

Quality Management System

Vector Informatik GmbH has ISO 9001:2010 certification. The ISO standard is a globally recognized standard.

1.1.2 Warranty

Restriction of warranty

We reserve the right to modify the contents of the documentation or the software without notice. Vector disclaims all liabilities for the completeness or correctness of the contents and for damages which may result from the use of this documentation.

1.1.3 Support

Need support?

You can reach our hotline by calling
+ 49 711 80670 3900
or send a problem report to embeddedSupport@de.vector.com.

1.1.4 Trademarks

Protected trademarks

All trademarks mentioned in this documentation and if necessary third party registered are absolutely subject to the conditions of each valid label right and the rights of particular registered proprietor. All trademarks, trade names or company names are or can be trademarks or registered trademarks of their particular proprietors. All rights which are not expressly allowed are reserved. If an explicit label

of trademarks, which are used in this documentation, fails, should not mean that a name is free of third party rights.

- > Outlook, Windows, Windows XP, Windows 2000, Windows NT, Visual Studio are trademarks of the Microsoft Corporation.

1.1.5 Example Code



Caution: The Start Application is only intended for illustrating an example of a possible BSW integration and BSW configuration.

The Start Application has not passed any quality control measures and may be incomplete. The Start Application is neither intended nor qualified for use in series production.

The Start Application as well as any of its modifications and/or implementations must be tested with diligent care and must comply with all quality requirements which are necessary according to the state of the art before their use.

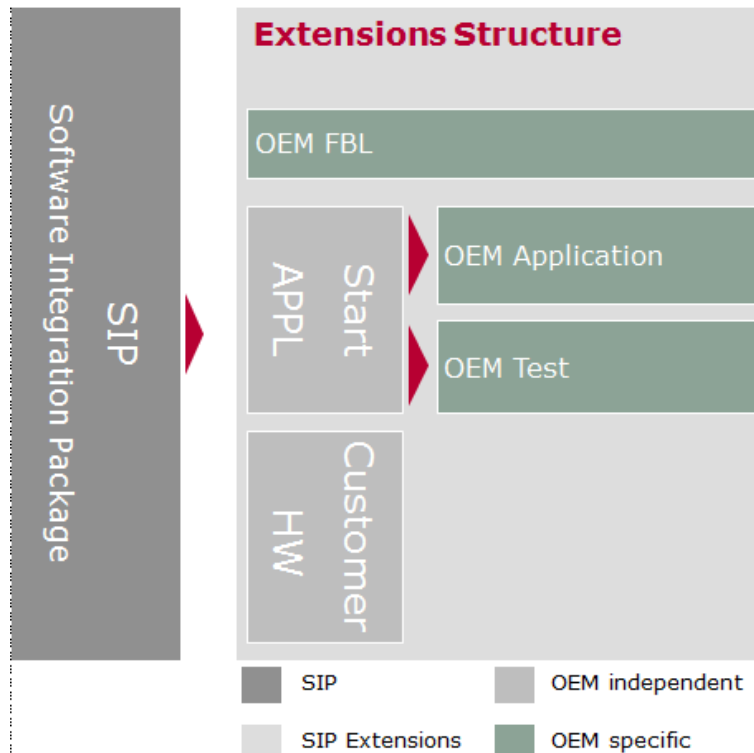
1.2 Overview

Welcome to Vector Start Application

The Vector Start Application offers you an easy starting point for your ECU project. It contains ECU-wise:

- > A configured BSW stack, containing your ordered modules
- > StartUp Code to get the stack up and running
- > An application in terms of a SWC, whose implementation demonstrates the functionality of the SIP, based on the Autosar clusters e.g. for communication, memory, and diagnostics.

Start Application Layer Illustration





Note: Additionally, the counterpart of the Start Application is provided in form of a CANoe setup, which lets you trigger certain cluster functionality via panels. The result of the interaction is also visualized in the panels.

The Vector Start Application is more than a Demo as it is based on your project specific input data for communication (EcuExtract / Fibex / Dbc) and diagnostics (ODX/EcuExtract). Moreover, you get a running system, where you can self-check the cluster functionality with the CANoe counterpart.

This way, the Vector Start Application can provide a starting point for your ECU project. It can be easily adapted to meet your ECU specific needs.

This document gives you an overview over the delivered application. It shows how the application demonstrates the functionality of different AUTOSAR clusters.



Caution: Although your project specific input data are used for the configuration of the BSW, there can be elements which are added or extended for test purposes. Those extensions are made to increase the test coverage and applicability of use cases. A common naming convention is applied and such elements can be identified based on the keyword 'StartApplication'.

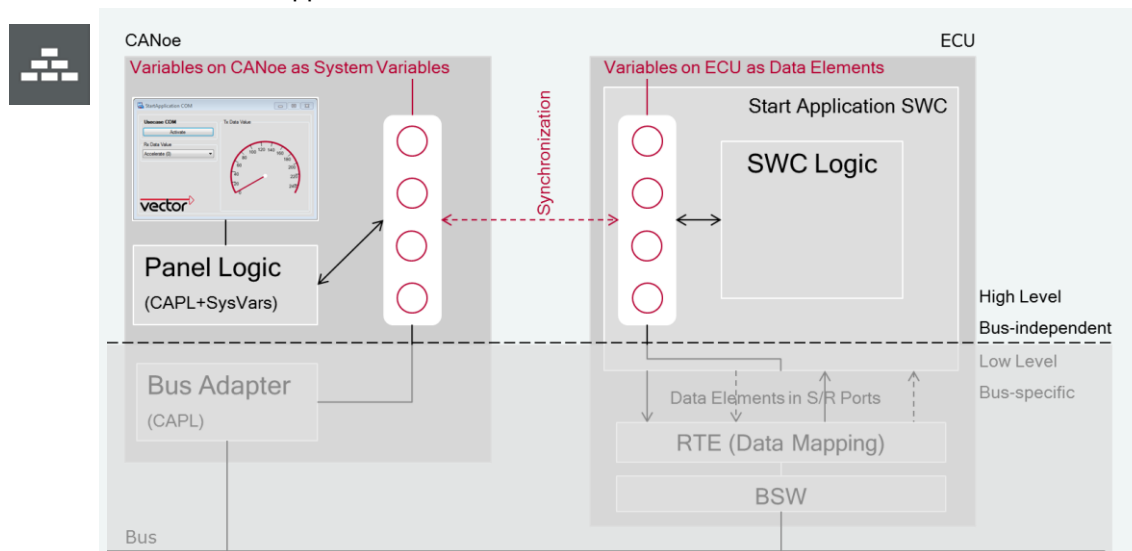
2 Bus Independent Functionality – High Level

This chapter guides you through the use cases of the delivered Vector Start Application. In all use cases you can interact with the application on ECU-side by using a CANoe counterpart, controlled with panels. The following sections explain the basic architecture as well as the use-case functionality on a bus-independent level.

2.1 Basic Architecture

Synchronization of variables between ECU and CANoe

The interaction between panels in CANoe and the SWC application on the ECU is based on 4 variables. To understand the basic functionality, it is only important to know that you can set and read variables in CANoe panels, which are synchronized with the SWC application via inter ECU communication.



Rx variables

There are two pairs of variables. One pair is set by CANoe and received by the application:

- > RxCtrl
- > RxData

Tx variables

The other pair is sent by the application and received by CANoe:

- > TxCtrl
- > TxData

CANoe variable handling

The CANoe variables are represented by CANoe system variables which are connected to certain control elements in the CANoe panels (panel logic). Each time a panel control element changes the RxCtrl or RxData variable, a system variable handler is activated, who propagates the new value to the application via a communication channel (bus adapter). If the application provides new values for the TxCtrl variable or the TxData variable, the bus adapter updates the system variables and system variable handler updates the panel controls (panel logic).

ECU variable handling

The Rx variables on application side are represented by data elements in either one or two receiver ports, depending on the mapped bus-specific communication element for the inter ECU communication. E.g. if the synchronization is done with COM signals, you might have two receiver ports, each representing one data element. On the other hand if the synchronization was done with COM signal groups, you might have only one receiver port, where you can receive the whole signal group and handle the data elements in the implementation by accessing the corresponding struct members. New values for the Tx variables can be propagated to CANoe via one or two sender ports. That means the bus-specific part on ECU-side is handled in the S/R ports, their mapping (RTE) and of course by the basic software (BSW).

For better understanding all following sections assume one port for each data element.

To see how the inter ECU communication is managed in this delivery refer to section 3.3

2.2 CANoe Version Dependencies

Features and API

The use cases which are shown in the context of the Start Application might possibly use CANoe functionalities and API calls which are not supported within your installed CANoe version. To reduce negative side effects the Start Application performs a CANoe version check and encapsulates code sections which cannot be executed.

Log information

In case a restriction is detected and parts of a use case or complete use cases might not work you will receive a corresponding information:

> Write Window:

Detailed information can be found in the Start Application Write Window at the very beginning after measurement start.

> Test Report:

The test script checks if the required CANoe version is used and reports a failure in case the CANoe version is not suitable for the test.

Dependencies

The following table contains a listing of use cases or parts of use cases for which the Start Application implementation has a CANoe version dependency. Use cases which are not listed below might require dedicated CANoe Options but have no additional CANoe version dependency.

	Functionality	Supported since
Common	All Start Application use cases	CANoe 8.2 Main Release
	CAN FD for Rx control signals	CANoe 8.5 SP6, CANoe 9.0 SP4
	SecOC used for control signals	CANoe 10 SP3
COM	CAN FD for Rx signals	CANoe 8.5 SP6, CANoe 9.0 SP4
	SecOC used for signals	CANoe 10 SP3
DIAG	Ethernet (DoIp)	CANoe 8.5 SP3
FBL	vFlash automation interface API	CANoe 8.5 SP3
NM	Ethernet (UDP Nm)	CANoe 8.5 Main Release

Panel Exception

If different CANoe versions are installed, it can happen that the use case panels in the CANoe configuration show an error message instead of the actual panel. In this case unregister the COM server that is currently registered. Afterwards register the COM server for the CANoe version which is used for the Start Application:

- > Check the CANoe_InstallDir environment variable
- > Go to the CANoe installation directory referenced in this environment variable
- > Execute RegisterComponents.exe /u
- > Go to the target CANoe version which should be used for the Start Application
- > Execute RegisterComponents.exe



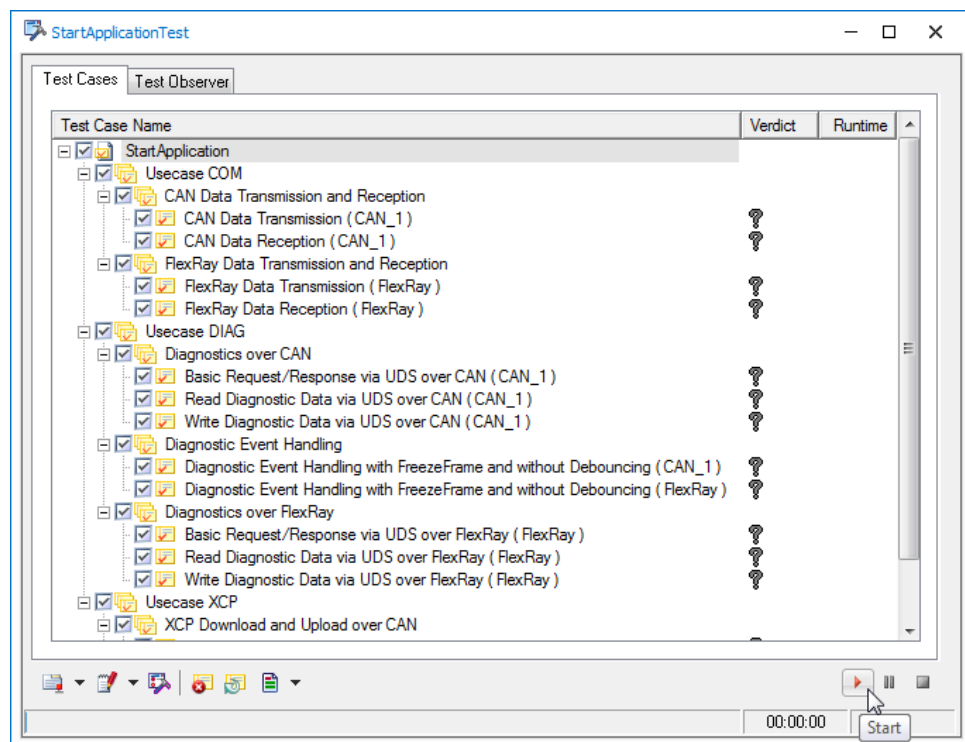
Note: The COM Server will be registered as part of the CANoe installation process. You can figure out the currently referenced COM Server by checking your environment variables. Typically, you can find one or more of the following variables:

- > CANoe_InstallDir
- > CANoe_InstallDir32
- > CANoe_InstallDir64

The CANoe_InstallDir variable references the installation directory of the currently registered COM Server.

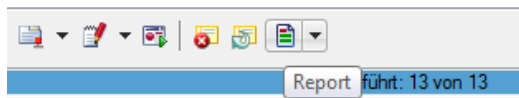
2.3 Getting Started

- CANoe configuration** Depending on the available use cases some additional manual steps must be performed to use the Start Application. More details can be found in the ReadMe.txt (CANoe\StartApplication\ReadMe.txt).
- Start desktop** The start desktop in the delivered CANoe configuration allows you to check all use cases automatically at once by executing a test script. That means you can quickly check if your system and your hardware setup basically works, without knowing the use cases' functionality in detail.
- Behavior** The test script triggers use case specific functionality in the corresponding testcases and verifies that the behavior of the ECU is as expected. The use case specific behavior is described in the following section.
- Execute the test script** In order to execute the test script, start the measurement and simply press the little **[Play]** Button in the test script window:



Check the test results

After the test run, you can see the results directly in the test script window or you can access the report by clicking the report symbol in the test script window:



HTML report

The HTML report contains all aggregated information about the test run:

Test Case Results

1		Usecase COM	
1.1		CAN Data Transmission and Reception	
1.1.1	TCASE-375705	CAN Data Transmission (CAN 1)	pass
1.1.2	TCASE-375706	CAN Data Reception (CAN 1)	pass
1.2		FlexRay Data Transmission and Reception	
1.2.1	TCASE-375707	FlexRay Data Transmission (FlexRay)	pass
1.2.2	TCASE-375708	FlexRay Data Reception (FlexRay)	pass
2		Usecase DIAG	
2.1		Diagnostics over CAN	
2.1.1	TCASE-478915	Basic Request/Response via UDS over CAN (CAN 1)	pass
2.1.2	TCASE-481982	Read Diagnostic Data via UDS over CAN (CAN 1)	pass
2.1.3	TCASE-483095	Write Diagnostic Data via UDS over CAN (CAN 1)	pass
2.2		Diagnostic Event Handling	
2.2.1	TCASE-379331	Diagnostic Event Handling with FreezeFrame and without Debouncing (CAN 1)	pass
2.2.2	TCASE-379331	Diagnostic Event Handling with FreezeFrame and without Debouncing (FlexRay)	pass
2.3		Diagnostics over FlexRay	
2.3.1	TCASE-478916	Basic Request/Response via UDS over FlexRay (FlexRay)	pass
2.3.2	TCASE-481983	Read Diagnostic Data via UDS over FlexRay (FlexRay)	pass
2.3.3	TCASE-483096	Write Diagnostic Data via UDS over FlexRay (FlexRay)	pass
3		Usecase XCP	
3.1		XCP Download and Upload over CAN	
3.1.1	TCASE-489438	XCP Download and Upload over CAN (CAN 1)	pass

By clicking on the links, you can access detailed information about all test steps of each test case.

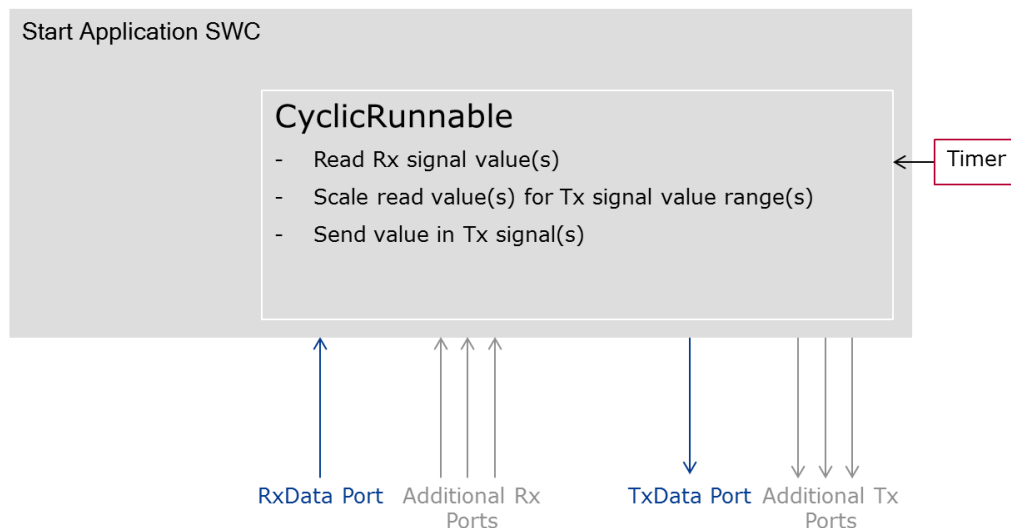
2.4 Use Case COM

Sender/Receiver on runnable level

The use case COM demonstrates the basic sending and receiving of data on application level. More precisely, it demonstrates the inter ECU communication using sender/receiver ports on runnable level. This behavior is represented by the following SWC design:

2.4.1 SWC

Structure



Behavior

The cyclic runnable is triggered each 250ms. Within this runnable, the data of the receiver port RxData is read. The received value is sent to the TxData port. In case the data type of RxData differs from the data type of TxData the value is scaled in order to use the full data type range.

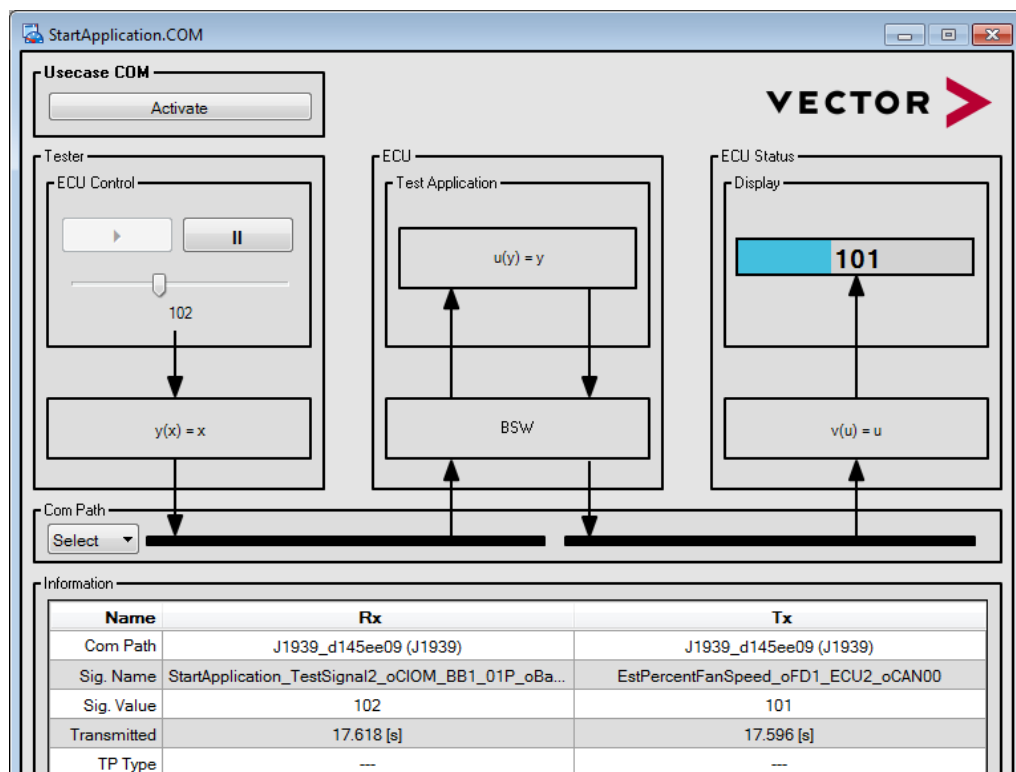
If you have a multi-bus ECU, you get additional ports – usually one Rx and one Tx Port per bus, given that you have adequate signals on those busses. In this case the cyclic runnable reads all Rx data ports and sends each value back on its corresponding Tx port.

If a certain bus has either adequate Tx signals or Rx signals, but not both, the missing path is handled on another bus if available.

2.4.2 CANoe

Panels

CANoe plays the counterpart of the SWC, and allows you to see and to influence the data element values, by synchronizing them with its system variables, as described in the previous chapter. The system variables are connected in this use case to the following panel:



The slider in the CANoe panel generates a sensor value and puts this value into a Rx signal. The application in the ECU reads this value and sends it back to CANoe in a Tx signal. The Tx signal value is displayed in the control meter in the panel and should be equal to the sensor value. In summary, this results in the following data path: ECU Control → Rx Signal → ECU → Tx Signal → Display. Note: For each link denoted by an arrow an adequate scaling function is used to ensure that the complete value range of the signal is used. If a signal has an array data type a single byte of the array is used without scaling. Instead the position of the used byte changes depending on the sensor value. In this case the panel shows tooltips with a more detailed explanation for each of the formulas.

You can start the sensor value generation by pressing the **[Play]** button. The slider value increases and decreases automatically. You can also pause the value generation and set the slider value manually.

If you have more than one bus, you can choose your communication path using the select box.

The section at the bottom contains a data grid which provides additional information about the selected communication path. For each direction (Rx and Tx) the corresponding channel name, the used signal with the scaled signal value and the timestamp when the signal was transmitted, are shown. Those information are provided for all communication paths, regardless of their bus-specific transmission type. The brackets behind the channel name contain additional information about the selected channel. In this case the classified bus system type J1939. Other entries can be CAN, LIN or FlexRay.

For certain scenarios the grid displays further bus-specific information in additional rows, e.g. there can be an additional row for J1939 which shows the transport protocol type like BAM or CMDT.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.



Expert knowledge: The CANoe panels themselves are static and communicate with system variables only. Thus, they are independent from your project specific input data for communication like the SWC. The “mapping” on CANoe side is done with CAPL code, see section 3.2

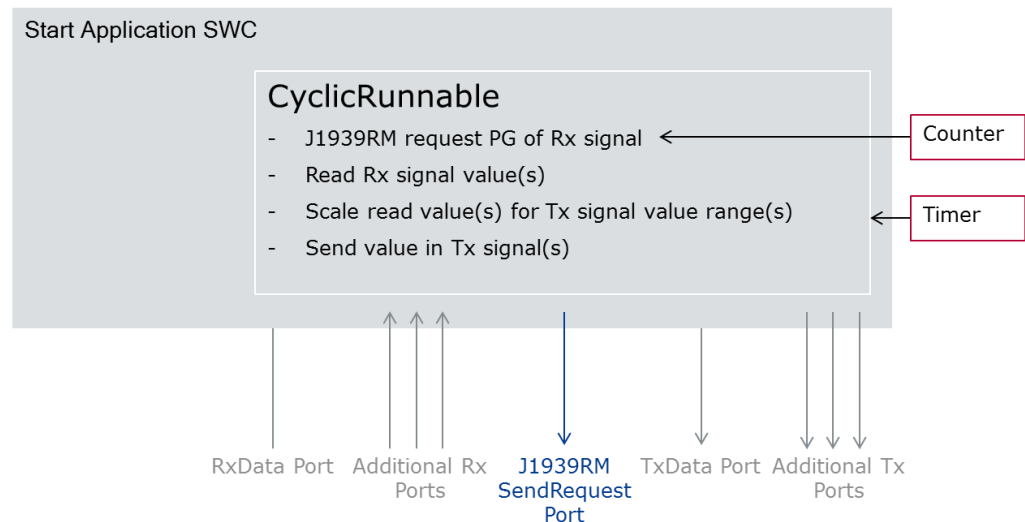
2.5 Use Case COM – J1939RM Extension

J1939 RM SendRequest port on runnable level

The use case J1939RM demonstrates the basic sending of a request and receiving of requested data on application level. More precisely, it demonstrates the inter ECU communication using J1939RM SendRequest port on runnable level. The J1939RM is integrated within the COM use case. The behavior is represented by the following SWC design:

2.5.1 SWC

Structure



Behavior

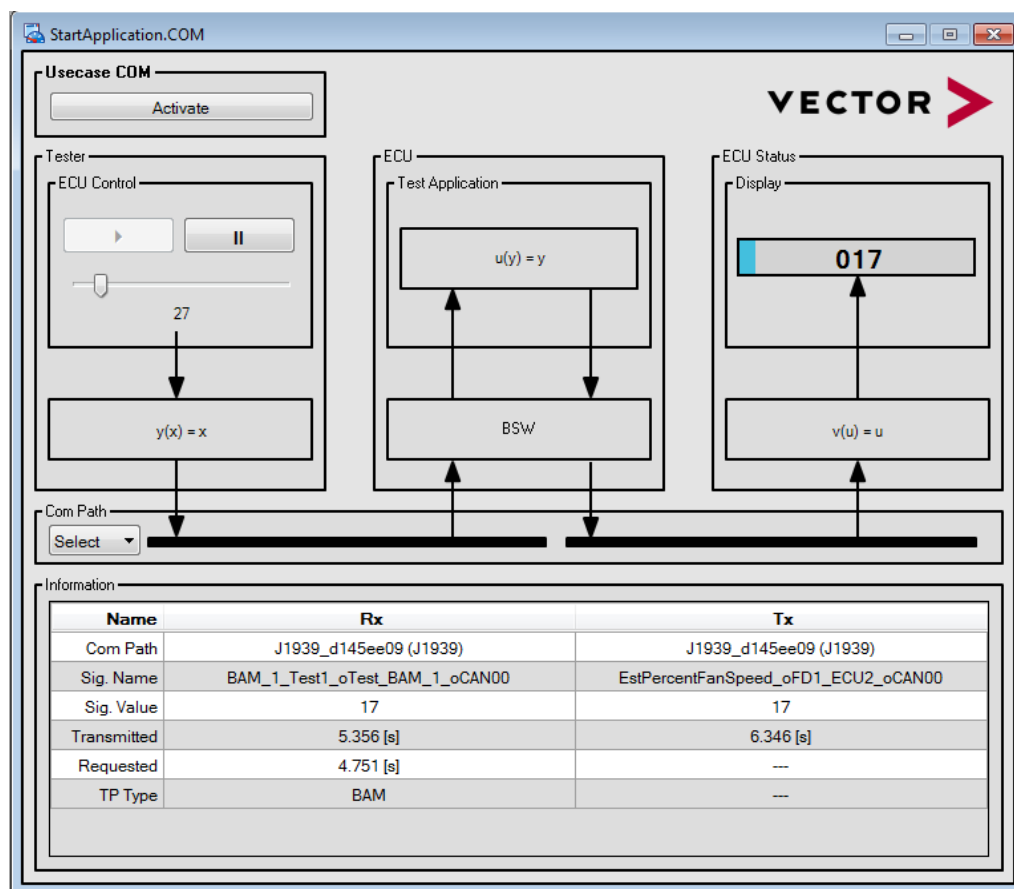
The cyclic runnable is triggered each 250ms. Within this runnable a counter is used to trigger the request of a ParameterGroup via SendRequest port of the J1939RM every 5s. The request is done for signals that are mapped to a ParameterGroup that is transmitted via transport protocol type BAM or CMTD. Afterwards the data of the receiver port RxData is read. The received value is sent to the TxData port. In case the data type of RxData differs from the data type of TxData the value is scaled in order to use the full data type range.

If you have a multi-bus ECU, the handling of the Rx and Tx data ports is done equivalent to the COM use case.

2.5.2 CANoe

Panels

Like in the original COM use case (see 0) CANoe plays the counterpart of the SWC, and allows you to see and to influence the data element values, by synchronizing them with its system variables.



The functionality of the Panel is identically to the original COM use case (see 0) except the update behavior of the signal values and the data grid content.

For signals that are mapped to a ParameterGroup that is transmitted via transport protocol type BAM or CMTD there will be a difference between the slider value and the signal value. When the **[Play]** button is used to automatically generate the input values the signal will be updated continuously but not transmitted on the bus. The signal transmission has to be requested by the ECU that means the signal is transmitted only on demand. This will also force an update of the data grid content. The signal value within the data grid corresponds to the input slider value when the request occurs. While the slider continuously increases those two values can be asynchronous.

For the J1939RM use case there are two channel specific rows within the data grid. The first row is the requested timestamp that corresponds to the reception of the ParameterGroup request message. The second row is the transport protocol type. For segmented multicast ParameterGroups the entry will be the BAM, for segmented unicast ParameterGroups the entry will be CMTD and all others have the value '---'.

2.6 Use Case COM – SecOC Extension

Message authentication

The COM use case extension for SecOC demonstrates message authentication using SecOC.

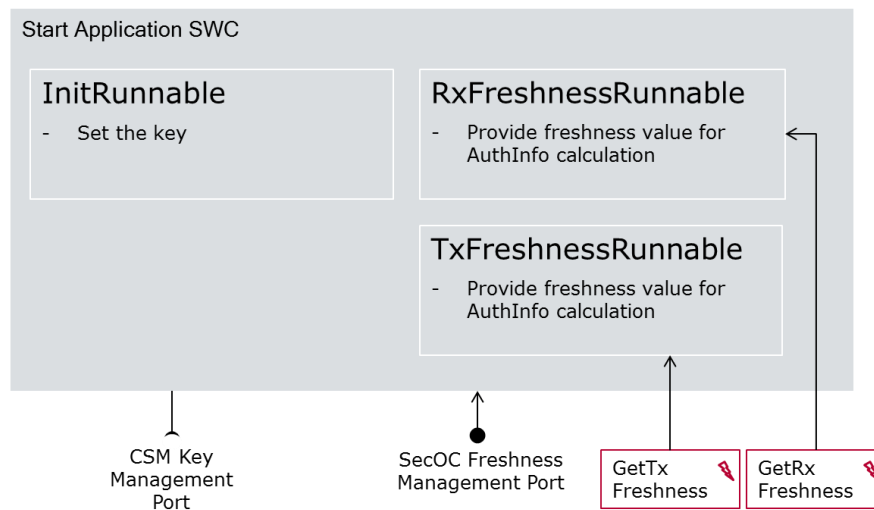
Vector Security Manager Configuration in CANoe

Start Application with SecOC requires a specific security profile in the Vector Security Manager which must be manually imported:

Open the Vector Security Manager from the CANoe ribbon "Tools" and import the file "Configuration\StartApplicationSecOCProfile.xml" located in the same path as your "StartApplication.cfg". Additional information can be found in the ReadMe.txt (CANoe\StartApplication\ReadMe.txt).

2.6.1 SWC

Structure



Behavior

This is an addition to the standard COM use case behavior for signals transmitted using SecOC.

The cyclic runnable is the same as pictured in 2.4.1 with the following behavior: Read the value of the Rx signal → Scale the value → Send the value using the Tx signal. For the SecOC handling the SWC provides a key and Freshness Values.

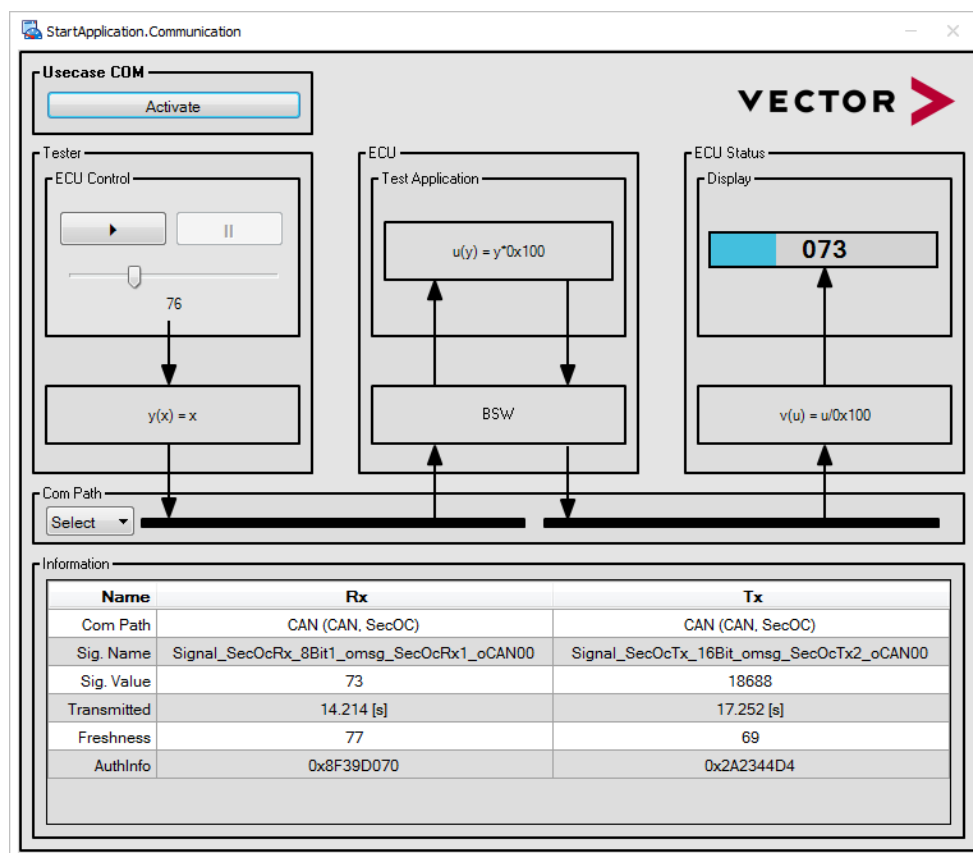
The Init Runnable is triggered after ECU start up and sets the key element bytes of the key and then sets the state of the key to valid.

The Freshness Runnables provide the Freshness Values used for AuthInfo calculation by the SecOC module. The Rx Freshness Runnable is called by the SecOC module before the AuthInfo calculation when a SecOC PDU was received. The Tx Freshness Runnable is called by the SecOC module before the AuthInfo calculation when a SecOC PDU shall be transmitted. The Start Application implements a simplified freshness value handling which always leads to a successful verification.

2.6.2 CANoe

Panels

Like in the original COM use case (see 0) CANoe plays the counterpart of the SWC, and allows you to see and to influence the data element values, by synchronizing them with its system variables.



The functionality of the panel is identically to the normal COM use case (see 0).

Signals which are transmitted in a secured PDU are marked with “SecOC” in the Com Path Selector and in the first row of the data grid.

The data grid contains two additional rows for all signals which are transferred in a secured PDU: The truncated freshness value and the authenticator (MAC). The shown values are part of the secured PDU which contain the selected signal.

2.7 Use Case DIAG

RDBI/WDBI on runnable level (DCM)

The use case DIAG demonstrates reading and writing of a specific application variable with the RDBI/WDBI diagnostic service. This is done via the “WriteData” and “ReadData” operations of a DcmData service port provided by the Start Application.

Setting the operation cycle and setting DTC to passed/failed (DEM)

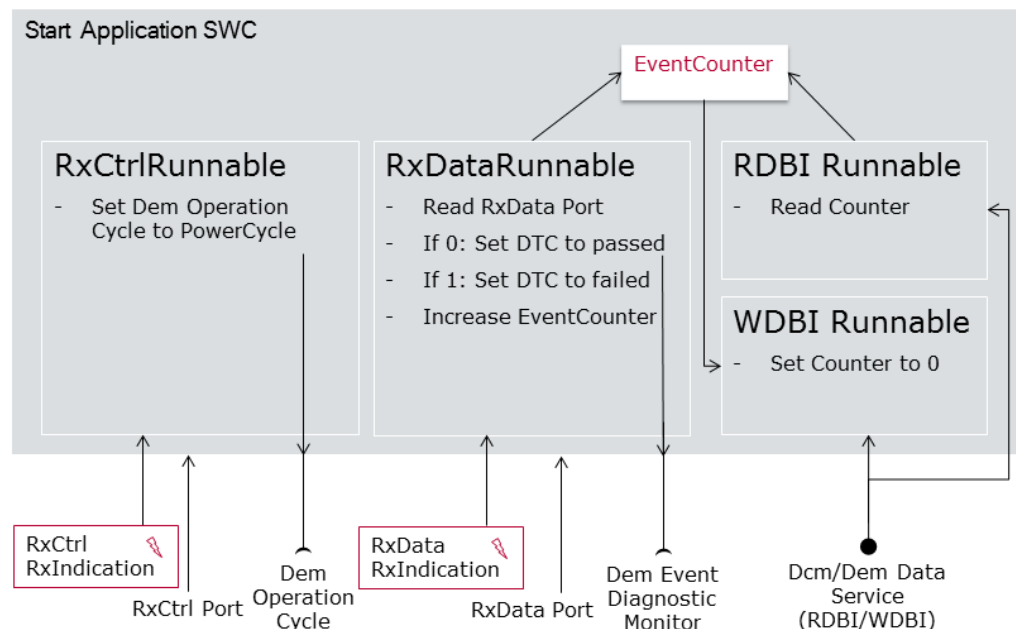
Further the DIAG use case comprises setting the DEM operation cycle via an OperationCycle port as well as setting a specific DTC to passed or failed via a DiagnosticMonitor port on application level. To call DEM services for setting the event status, inter ECU communication via sender/receiver ports is used as in the COM use case.

Additionally, when the event is set to failed, the current value of a specific application variable is written to a data snapshot. This snapshot can be read by the RDTCCSSBDTC service and can be cleared by the service CDTCl. Those two services are BSW-only and have no application part.

This behavior is represented by the following SWC design:

2.7.1 SWC

Structure



Behavior

On the right hand side, you can see the runnables triggered by the DCM when requesting RDBI or WDBI service for a predefined DID (0x55) over diagnostic protocol. In case of RDBI the value of the DID signal is set to the value of the EventCounter and sent with the diagnostic response. In case of WDBI the value of the counter is set to '0' regardless of the DID's value.

The left hand runnable and the runnable in the middle are triggered by incoming data. Within those runnables the DEM service ports are called.

In the middle, there is the RxData reception runnable, triggered on reception of a value for RxData. Depending on the received value the DTC is set to failed or passed with the “SetEventStatus” operation of the DiagnosticMonitor Port. Each time the event is set to either passed or failed, the event counter is increased. Setting the DEM Event to fail will additionally store the counter in a DEM snapshot. The snapshot value can be read by RDTCCSSBDTC and reset by CDTCl.

If the RxCtrl runnable is triggered by receiving data on the RxCtrl port, the operation cycle is set with the SetOperationCycleState operation of the OperationCycle port.

2.7.2 CFG5

The service ports need to be mapped to their counterpart within the DEM/DCM module. Therefore, specific DEM/DCM containers were configured in the BSW.

The following DID was added:

The first screenshot shows the configuration of the Data Identifier (DID) **DID_StartApplication**. The left pane shows the project tree with **Data Identifiers (DIDs)** expanded, showing **DID_StartApplication** and **Data Objects** (containing **Data_DID_StartApplication**). The right pane shows the configuration for **DID_StartApplication**:

- Name: **DID_StartApplication**
- ID: **0x55** (hex)
- Used: ☒
- DID Info: **DidInfo_DID_StartApplication** [...]
- DID Info** section:
 - DID Info Name: **DidInfo_DID_StartApplication**
 - Dynamically Defined: ☒
 - IO Enable Mask Size: (dec)
 - Read** ☒
 - Write** ☒
 - Control** ☐
 - Define** ☐
 - External Response-On-Event** ☐
- DID Signals** table:

DcmDspDidSignals	Dsp Did Data Pos	Dsp Did Data Ref
DcmDspDidSignal	0	Data_DID_StartApplication

The second screenshot shows the configuration of the Data Object **Data_DID_StartApplication**. The left pane shows the project tree with **Data Objects** expanded, showing **Data_DID_StartApplication**. The right pane shows the configuration for **Data_DID_StartApplication**:

- Name: **Data_DID_StartApplication**
- Size: **16** (dec)
- Type: **UINT16**
- Use Port: **USE_DATA_SYNC_CLIENT_SERVER**

A snapshot data record for DEM was configured and connected to the DID:

The screenshot shows the configuration of a Snapshot Record for the DID. The left pane shows the project tree with **Snapshot Records** expanded, showing **DemFreezeFrameClass_StartApplication** and **Data Identifiers (DIDs)** (containing **DemDidClass_StartApplication**). The right pane shows the configuration for **DemDidClass_StartApplication**:

- Name: **DemDidClass_StartApplication**
- Did Identifier: **0x55** (hex)
- Data Objects** section:
 - Port: **CBReadData_DemDataElementClass_StartApplication, Data Size [byte]: 2**

Start

Events

- DEM_EVENT_StartApplication

DEM_EVENT_StartApplication

Name: DEM_EVENT_StartApplication

Event Kind: SWC

Monitor Port: Event_DEM_EVENT_StartApplication

Info Port: EvtInfo_DEM_EVENT_StartApplication

Event Significance: [dropdown]

Event Priority: 2 dec

Enable Condition Group Ref: [dropdown]

Storage Condition Group Ref: [dropdown]

Operation Cycle Ref: PowerCycle [dropdown]

Failure Cycle Ref: [dropdown]

Failure Cycle Counter Threshold: [dropdown]

Event Destination: DEM_DTC_ORIGIN_PRIMARY_MEMORY

DTC

Select DTC...

UDS DTC: 0x482BFF hex

DTC Group: [dropdown]

DTC Severity: [dropdown]

DTC Functional Unit: [dropdown]

Afterwards the DEM module offers service ports for this setting events and the operation cycle, which can be connected in the “ECU Software Components” comfort view:

Port Prototype	Connected Component Prototy	Connected Port Prototype
PpDemOpCycle_Power	Dem	OpCycle_PowerCycle
PpDiagnosticMonitor_DEM_EVENT_StartApplication	Dem	Event_DEM_EVENT_StartApplication

Here the Start Application is the client of this provided service.

Since the DID was also configured to have WDBI/RDBI, the DEM/DCM modules expect those services to be offered by the Start Application:

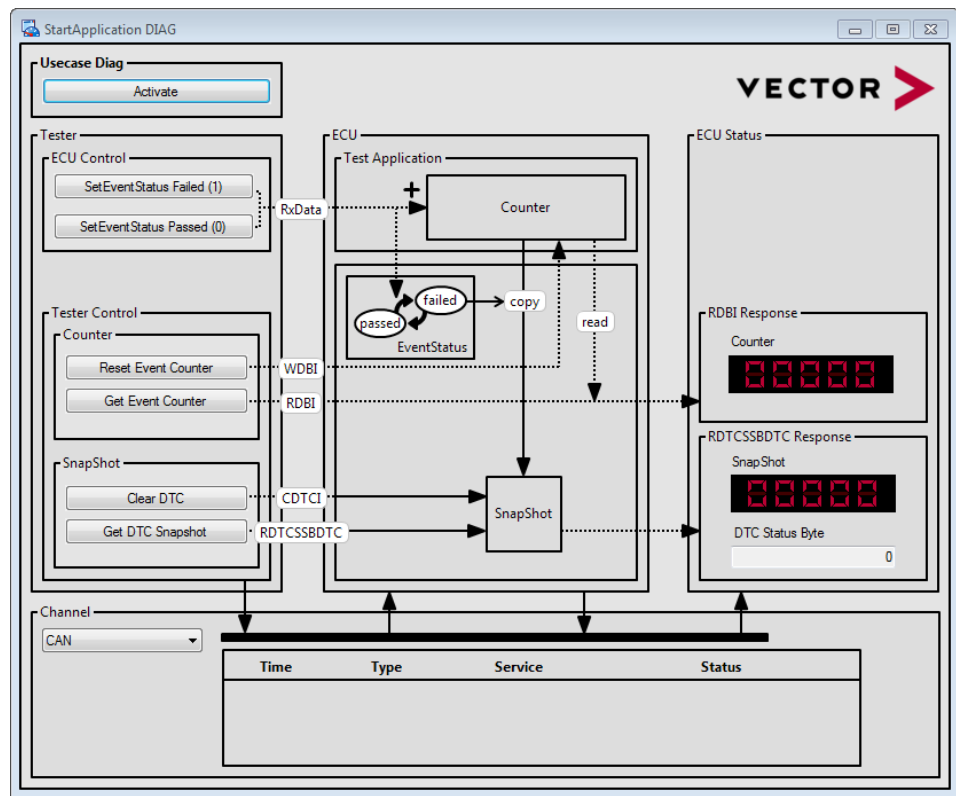
Port Prototype	Connected Component Prototy	Connected Port Prototype
PpDcmDataService_DID_StartApplication	Dcm	DataService_Data_DID_StartApplication
PpDemDataService_DemDataElementClass_StartA...	Dem	CBReadData_DemDataElementClass_StartApplication

Here the Start Application is server for a required service.

2.7.3 CANoe

Panels

In CANoe the application can be controlled and visualized via variable synchronization and diagnostic messages, which can be sent and received with CANoe panels, as in the COM use case.



Example: You can set the DEM event to passed or failed by using the two buttons in the ECU Control area. Those buttons trigger the transmission of the RxData variable with either the value '0' or '1'.

You can request different diagnostic services with the buttons in the Tester Control area. You can see if a request was handled successful in the ECU by looking at the data grid field at the bottom. The channel over which the diagnostic requests are sent can be selected using the channel drop-down list.

You can get the current counter value via RDBI by pressing the button **[Get Event Counter]**. Pressing this button triggers the transmission of a RDBI diagnostic request. If the response is positive, the read value is displayed in the RDBI Response field.

In the same manner, you can reset the counter in the ECU via WDBI. If you reset the counter via WDBI, do not forget to get the counter value afterwards via RDBI in order to see a change in the CANoe controls.

Setting the DEM Event to fail will additionally store the counter in a DEM snapshot. The snapshot value can be read by RDTCSBBDTC and reset by CDTCI in the same manner as RDBI/WDBI.

Additionally, for the RDTCSBBDTC service, the status byte of the response is visualized in "DTC Status Byte".

If you clear the snapshot data, do not forget to get the snapshot data afterwards in order to see a change in the CANoe controls.



Expert knowledge: In case a diagnostic service is not available in the configuration or in case it is configured with a specific security level the grid in the panel will show a negative response for those requests.

If a diagnostic service is configured with a specific session level then the required session will be requested before each request of the affected service.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

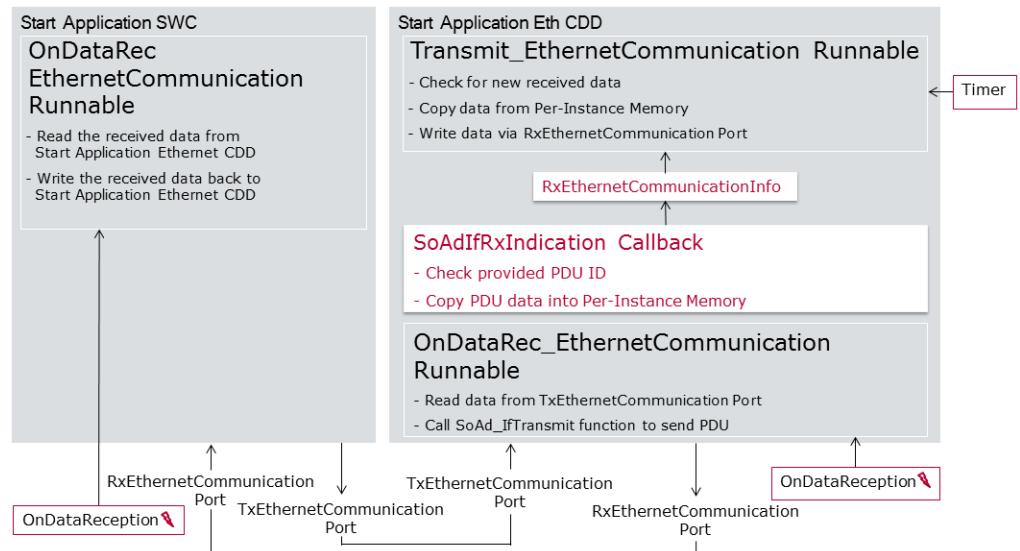
2.8 Use Case ETH

UDP data
transmission/
reception over
Ethernet

The use case ETH demonstrates the basic transmission and reception of UDP data over Ethernet. More precisely, a CDD is used to have a connection between the SoAd module and the application. This behavior is represented by the following SWC design:

2.8.1 SWC

Structure



Behavior

On the right-hand side, there is a SoAdIfRxIndication callback as entry point of the data processing. This callback is called after reception of a PDU. In case the PDU identifier matches the expected ID, the corresponding data is copied into a PIM. The Transmit_EthernetCommunication runnable above is called every 500ms. Inside the runnable the PIM is checked for new data. In case the new data is available it is copied from the PIM and written to the RxEthernetCommunication port.

The OnDataRec_EthernetCommunication runnable on the left side is triggered by the OnDataReception event. The received data is readout from the RxEthernetCommunication port and written back to TxEthernetCommunication port.

Finally, the data which is written by the Start Application SWC is handled by the OnDataRec_EthernetCommunication runnable on the right side. The data read from the TxEthernetCommunication port and is passed to the SoAd via the SoAd_IfTransmit function call.

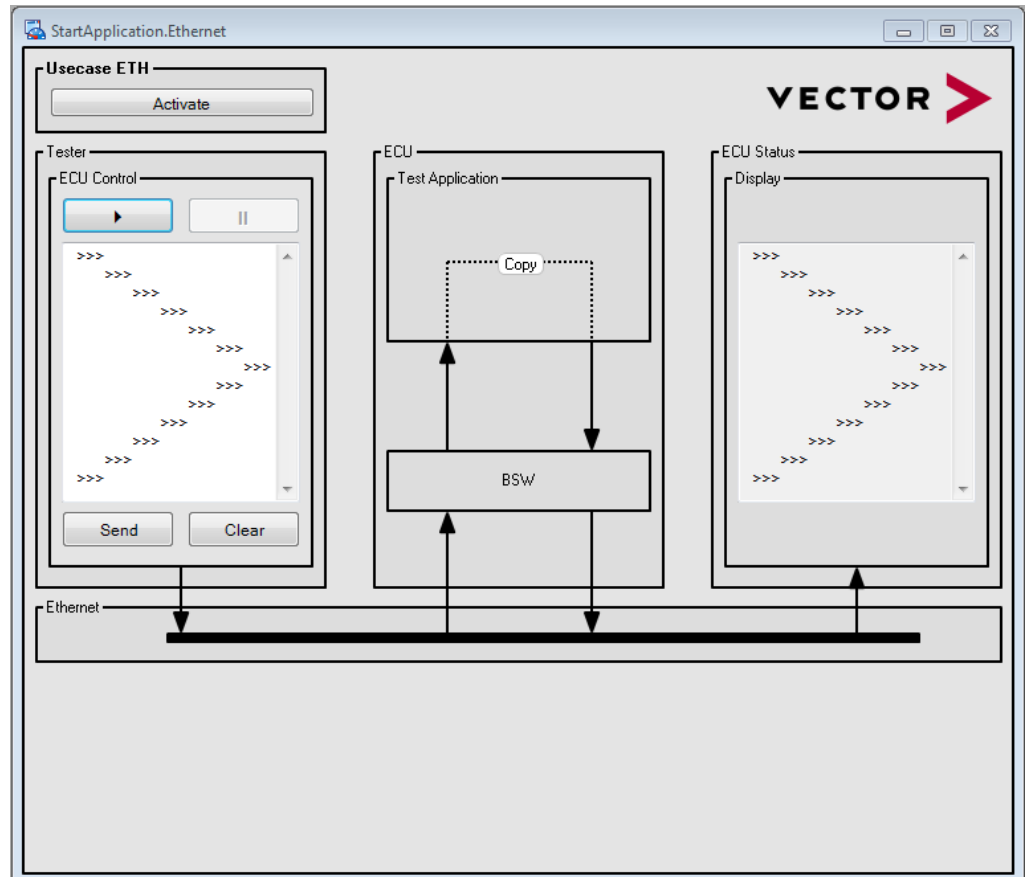


Note: There can be additional runnables and PIM implementations within the Start Application Ethernet CDD in case the control communication is handled via Ethernet.

2.8.2 CANoe

In CANoe, the application can be controlled via system variables which are connected with the following CANoe panel:

Panels



After the activation of the use case a predefined character sequence is loaded into the text box on the Tester side. The **[Play]** button activates the auto-play mode where the character sequence is shifted to the right, wrap-around and restarts on the left side of the text box. The **[Pause]** button can be used to stop the auto-play mode and modify the content of the text box manually. The auto-play mode itself or pressing the **[Send]** button triggers the transmission of the text box content via a PDU on the Ethernet channel. The UDP data is handled by the BSW and Start Application Eth CDD and forwarded to the Test Application. The Test Application in the middle copies the data and writes them back to the Ethernet channel. The resulting PDU data is displayed in the text box of the ECU Status on the right side.

The **[Clear]** button can be used in the pause mode to clear the content of the textbox on the Tester side.

Note: The maximum length of the PDU payload is 508 bytes and an empty PDU is not transmitted.



Use case activation

The **[Activate]** button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

2.9 Use Case FBL

Reprogramming of ECU

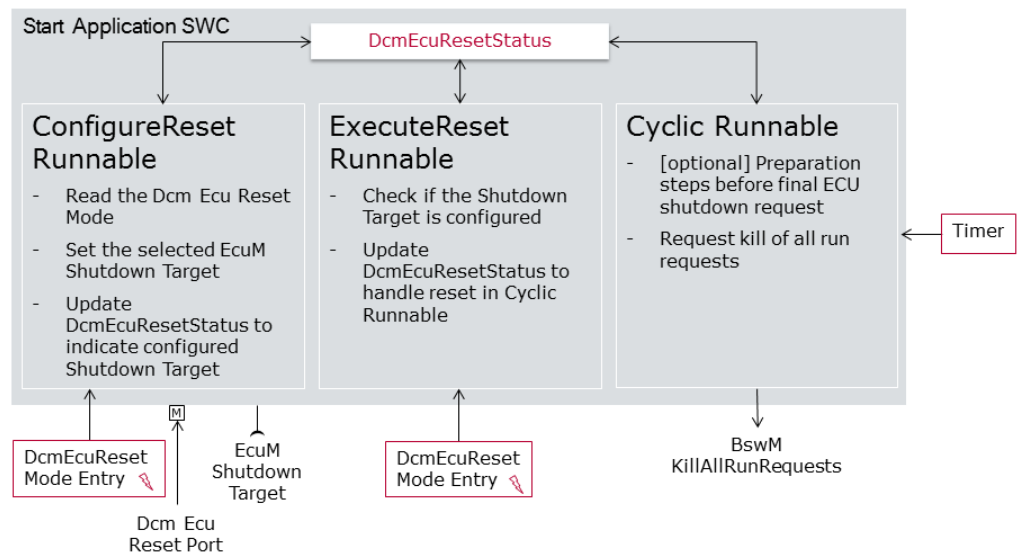
The FBL use case demonstrates that the ECU software binary can be flashed to the ECU.

vFlash

vFlash uses flash projects to save and exchange the configuration required for programming individual ECUs. Flash projects contain all relevant settings and reference the data files to be programmed. The Start Application uses in CANoe the automation interface of vFlash (vFlashNodeLayer.dll) for the reprogramming of the ECU software binary. Additional information can be found in the ReadMe.txt (CANoe\StartApplication\ReadMe.txt).

2.9.1 SWC

Structure



Behavior

The **ConfigureReset** runnable is the entry point for the reset processing from application point of view. It is triggered based on the **DcmEcuReset Mode Entry**. The **Dcm Ecu Reset** port value is readout and in case no active reset is configured previously the **EcuM Shutdown Target** Port is written. The **DcmEcuResetStatus** PIM is updated and indicates that a reset is configured. This PIM is used to handle the reset processing spread over the three runnables.

The **ExecuteReset** runnable is triggered by the **DcmEcuReset Mode Entry** too. In case the **EcuM Shutdown Target** was configured before an update of the **DcmEcuResetStatus** will be done to handle the reset processing within the **Cyclic** runnable.

The **Cyclic** runnable is triggered each millisecond. Here time-consuming operations can be done before the final shutdown is requested via the **BswM KillAllRunRequests** port. Therefore, a small state machine is implemented based on the **DcmEcuResetStatus** PIM. An example for time-consuming operations could be writing to the **NvM** and wait for the finished write operation.

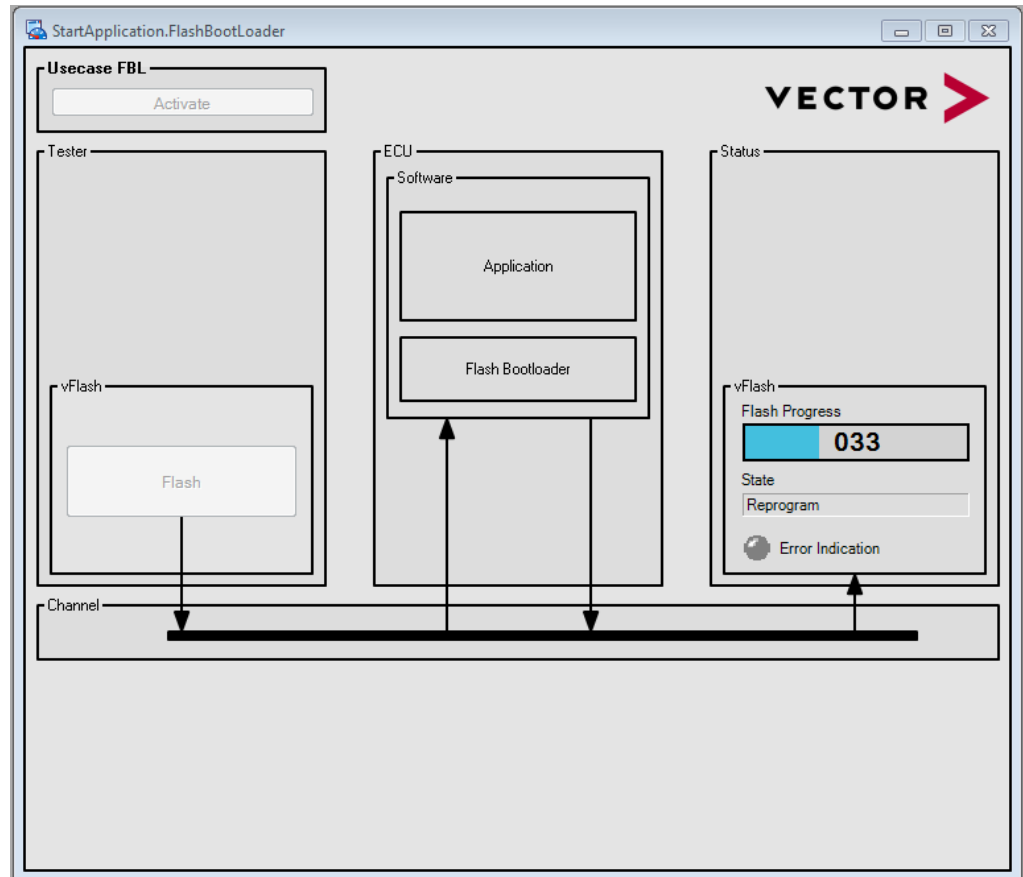


Note: The shutdown request via the **BswM KillAllRunRequests** port can already be triggered within the **ExecuteReset** runnable. The intention of the provided example implementation is to show how to handle operations that are not completed within a single function call.

2.9.2 CANoe

In CANoe, the application can be controlled via system variables which are connected with the following CANoe panel:

Panels



The **[Flash]** button on Tester side initiate the reprogramming of the ECU software binary via the vFlash automation interface. vFlash sends Diagnostic Service Requests to the ECU. The ECU software handles those requests and realize the shutdown preparation within the application. After completion of the shutdown preparation the ECU will perform the reset and restart in the Flash Bootloader. The Flash Bootloader realize the reprogramming of the ECU software binary. Finally, the ECU restarts with the new flashed ECU software. The **[Flash]** button is disabled while the reprogramming is running and is enabled after completion again.

On the right-hand side, the vFlash Status is visualized. The State contains the information about the current processing step that is done. While the reprogramming is running the Flash Progress bar shows the current progress in percentage. In case the vFlash automation API reports an error, the Error Indication LED is switched on. Further details can be found in the CANoe write window or test report.



Note: vFlash uses flash projects to save and exchange the configuration required for programming individual ECUs. Flash projects contain all relevant settings and reference the data files to be programmed. The result is a vflashpack file which is created in vFlash and used for the vFlash automation interface in CAPL.

Use case activation

The **[Activate]** Button is not required for the FBL use case. The reprogramming of the ECU software binary is driven by Diagnostic Services and the vFlash automation interface.

2.10 Use Case J1939DIAG

Setting the operation cycle and setting DTC to passed/failed (DEM)

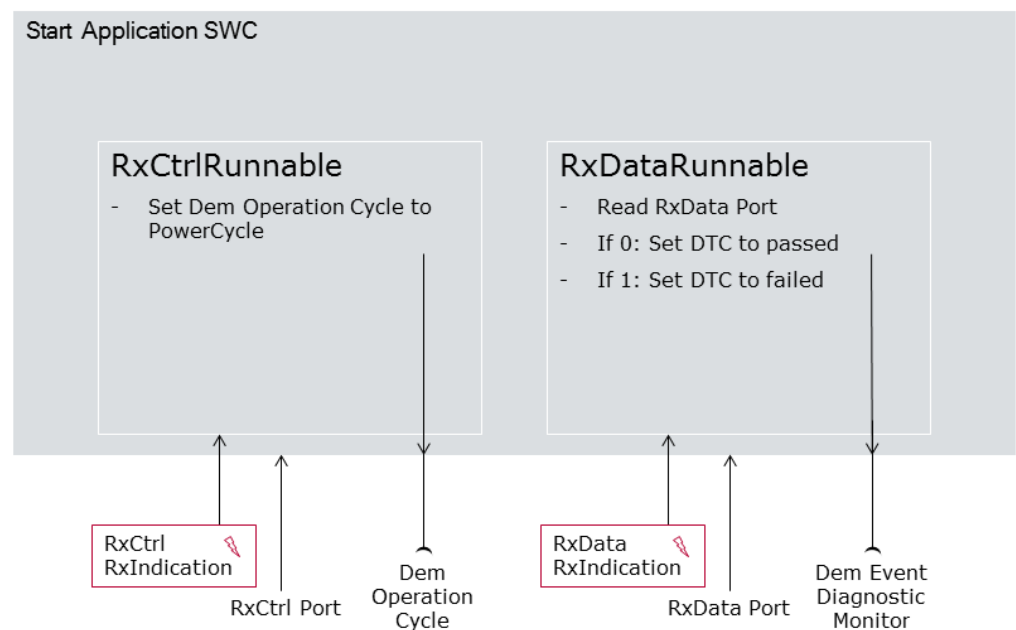
The use case J1939DIAG demonstrates setting the DEM operation cycle via an OperationCycle port as well as setting a specific DTC to passed or failed via a DiagnosticMonitor port on application level. To call DEM services for setting the event status, inter ECU communication via sender/receiver ports is used as in the COM use case.

The BSW handles the Active DTC occurrence counter and the Previously Active DTC occurrence counter by its own.

This behavior is represented by the following SWC design:

2.10.1 SWC

Structure



Behavior

The two runnables are triggered by incoming data. Within those runnables the DEM service ports are called.

On the right-hand side, there is the RxData reception runnable, triggered on reception of a value for RxData. Depending on the received value the DTC is set to failed or passed with the "SetEventStatus" operation of the DiagnosticMonitor Port. Each time the status switches from passed to failed, the BSW will increment the Active DTC occurrence counter. A switch of the status from failed to passed will trigger an update of the Previously Active DTC occurrence counter.

If the RxCtrl runnable is triggered by receiving data on the RxCtrl port, the operation cycle is set with the SetOperationCycleState operation of the OperationCycle port.



Expert knowledge: Depending on the configuration the following diagnostic messages are handled by the BSW:

- DM1 for cyclic transmission of the Active DTC occurrence counter
- DM2 for requesting the Previously Active DTC occurrence counter
- DM3 to reset / clear the Previously Active DTC occurrence counter

2.10.2 CFG5

The service ports need to be mapped to their counterpart within the DEM module. Therefore, specific DEM containers were configured in the BSW.

The screenshot shows the 'Diagnostic Events' configuration window. The left pane shows a tree view with 'All' > 'Events' > 'DEM_EVENT_StartApplication'. The right pane shows the configuration for this event:

- Name: DEM_EVENT_StartApplication
- Event Kind: SWC
- Monitor Port: Event_DEM_EVENT_StartApplication
- Info Port: EvtInfo_DEM_EVENT_StartApplication
- Event Significance: (dropdown)
- Event Priority: 2
- Enable Condition Group Ref: (dropdown)
- Storage Condition Group Ref: (dropdown)
- Operation Cycle Ref: PowerCycle
- Failure Cycle Ref: (dropdown)
- Failure Cycle Counter Threshold: (dropdown)
- Event Destination: DEM_DTC_ORIGIN_PRIMARY_MEMORY

Below these fields, there are sections for 'DTC' and 'Callbacks'.

DTC

Select DTC...

- UDS DTC: 0x482BFF
- DTC Group: (dropdown)
- DTC Severity: (dropdown)
- DTC Functional Unit: (dropdown)

Callbacks

Afterwards the DEM module offers service ports for the event setting and the operation cycle, which can be connected in the “ECU Software Components” comfort view:

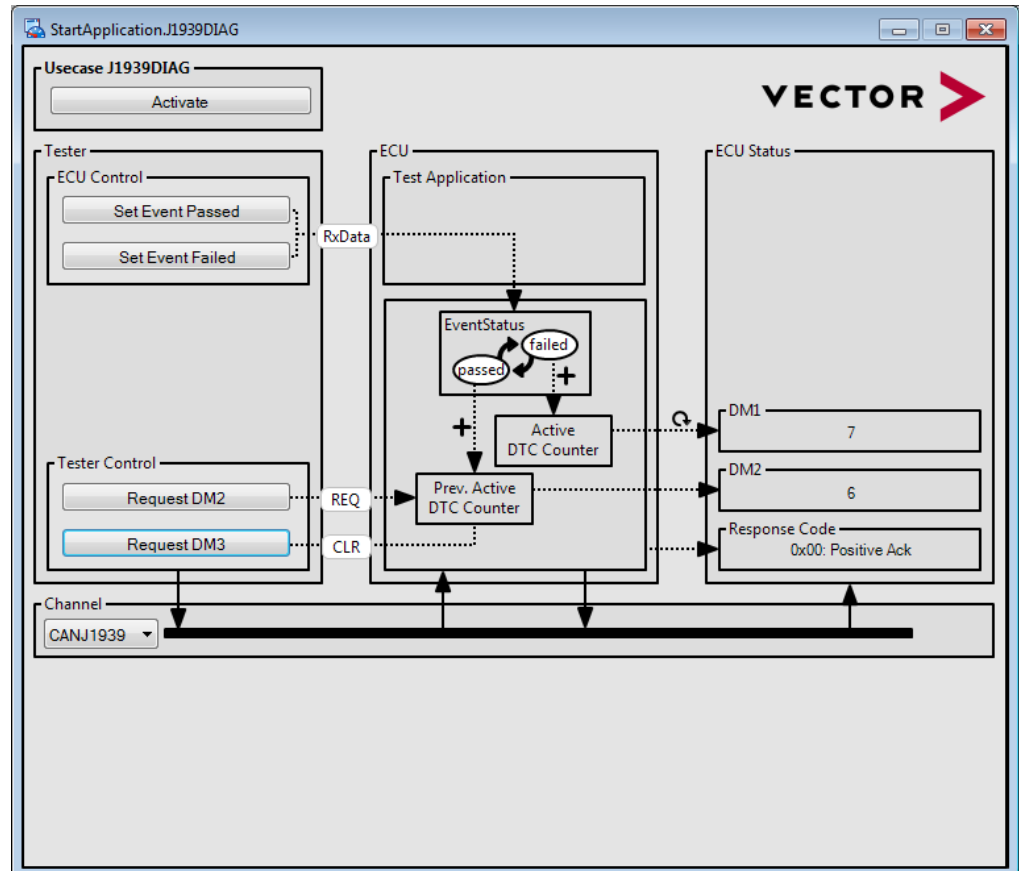
Port Prototype	Connected Component Prototy	Connected Port Prototype
PpDemOpCycle_Power	Dem	OpCycle_PowerCycle
PpDiagnosticMonitor_DEM_EVENT_StartApplication	Dem	Event_DEM_EVENT_StartApplication

Here the Start Application is the client of this provided service.

2.10.3 CANoe

In CANoe the application can be controlled via system variables which are connected with the following CANoe panel:

Panels



The ECU Control area buttons **[Set Event Passed]** and **[Set Event Failed]** provide the possibility to set the DEM Event Status to passed or failed via the RxData signal.

The buttons **[Request DM2]** and **[Request DM3]** in the Tester Control area trigger the requesting of the DM2 to get the Previously Active DTC occurrence counter or DM3 to clear / reset the Previously Active DTC occurrence counter.

The processing of the DEM Event Status and the request of diagnostic messages is depicted in the middle section of the Panel.

On the right-hand side the resulting ECU Status information is displayed. The DM1 area shows the Active DTC occurrence counter contained in cyclic DM1 or "DTC not found in DM1" when observed DTC is not part of DM1. Switching the event status from passed to failed will update this ECU Status content. The DM2 area shows the Previously Active DTC occurrence counter contained in requested DM2 or "DTC not found in DM2" when observed DTC is not part of DM2. The last content of the ECU Status area is the Response Code of the DM3 request.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

2.11 Use Case MEM

Read/Write block operation of NvM Service Port on runnable level

The use case MEM demonstrates the basic writing and reading to/from persistent memory. There are two different NvM blocks configured to demonstrate the usage of the operations “ReadBlock” and “WriteBlock” of a client/server NvM service port on runnable level. For each NvM block a NvMNotifyJobFinished port is provided by the Start Application to receive the “JobFinished” callback.

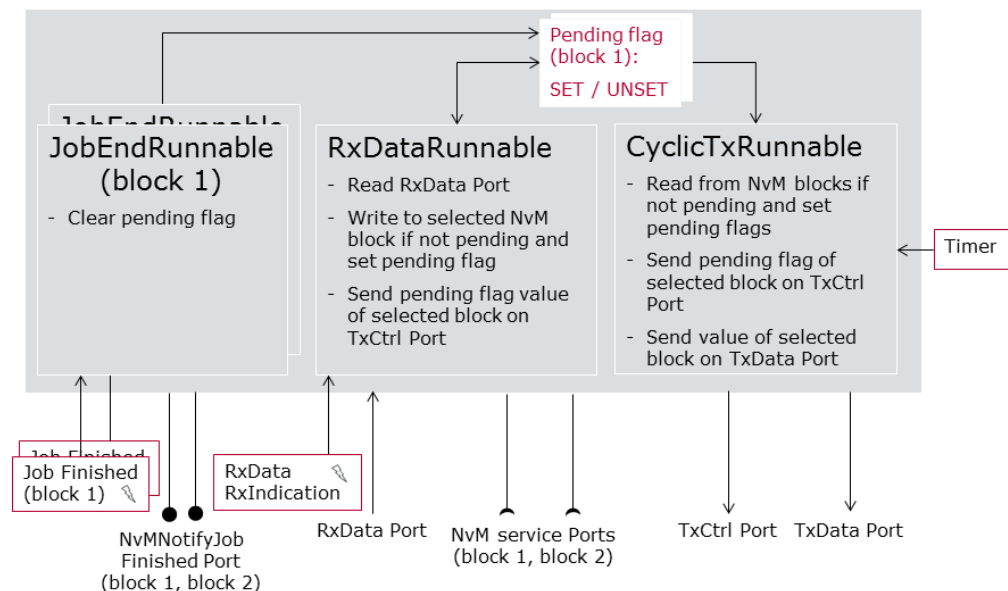
The inter ECU communication via sender/receiver ports is used to

- ... set the data to be stored
- ... receive the stored data
- ... receive the job status

The MEM use case behavior is represented by the following SWC design:

2.11.1 SWC

Structure



Behavior

On the right-hand side, there is a cyclic runnable triggered each 250ms. Within this runnable, data is sent on the sender ports TxData and TxCtrl on each call of the runnable. The value for TxData is read from the selected NvM block via the “ReadBlock” operation of the corresponding NvM service port. The value for TxCtrl represents the current value of the pending flag for the selected NvM block. If an operation is pending on a NV block, the “ReadBlock” operation for this block is not executed in the current runnable call and retried in the following calls.

In the middle, there is a data reception runnable, triggered on reception of a value for RxData. The received value is written to the selected NvM block via the “WriteBlock” operation of the corresponding NvM service port. If an operation is pending on the NvM block, the “WriteBlock” operation is not executed.

On the left-hand side, there are two job end runnables (block 1, block 2), triggered each time an operation on the corresponding NvM block is finished. The runnable clears the pending flag and enables upcoming operations.



Expert knowledge: The pending flag is a variable with more than two states. It can distinguish between the pending and finished states for the write and read operation separately.

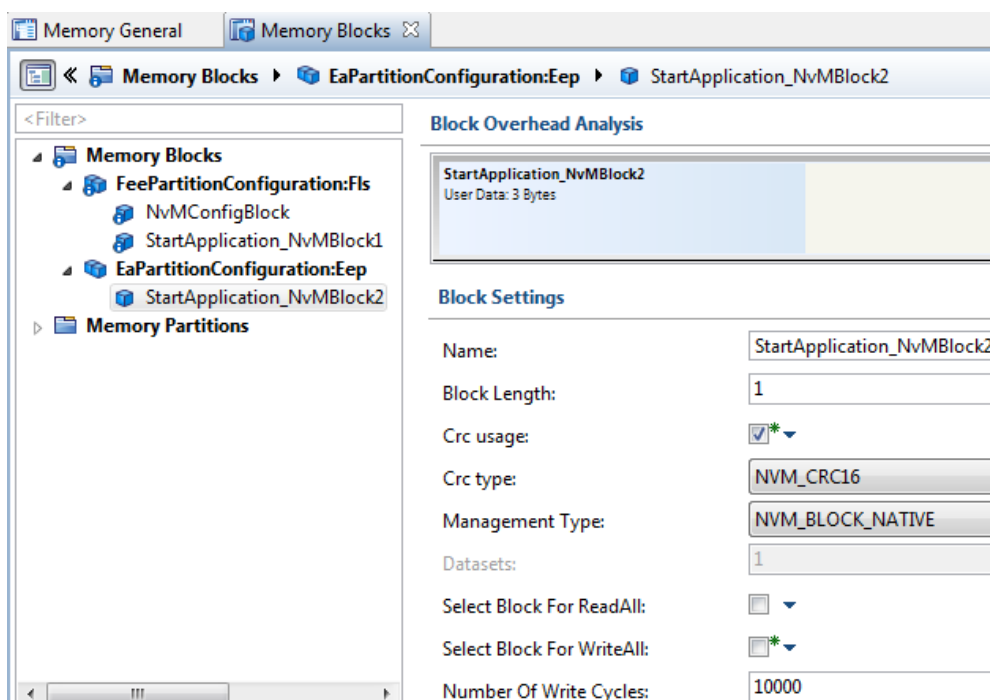
To prevent the cyclic runnable from blocking the write operation by executing the read operation each 250ms, the read operation is only executed if a write operation was finished before or if the use case was activated.

To prevent the RxData runnable from blocking the read operation, the write operation is only executed if a read operation finished before.

2.11.2 CFG5

NvM Block

The service ports need to be mapped to their counterpart within the NvM module. Therefore, there two specific NV blocks ('StartApplication_NvMBlock1', 'StartApplication_NvMBlock2') was configured in the BSW.



Since these blocks was configured to have service ports the NvM module offers NvM service ports for them, which can be connected in the "ECU Software Components" comfort view:

Service mapping

Port Prototype	Connects	Connected Port Prototype
PpPS_StartApplication_NvMBlock1	NvM	PS_StartApplication_NvMBlock1
PpPS_StartApplication_NvMBlock2	NvM	PS_StartApplication_NvMBlock2

Here the Start Application is the client for the provided services.

Since the blocks was also configured to have job end callbacks, the NvM module expects services to be offered by the Start Application:

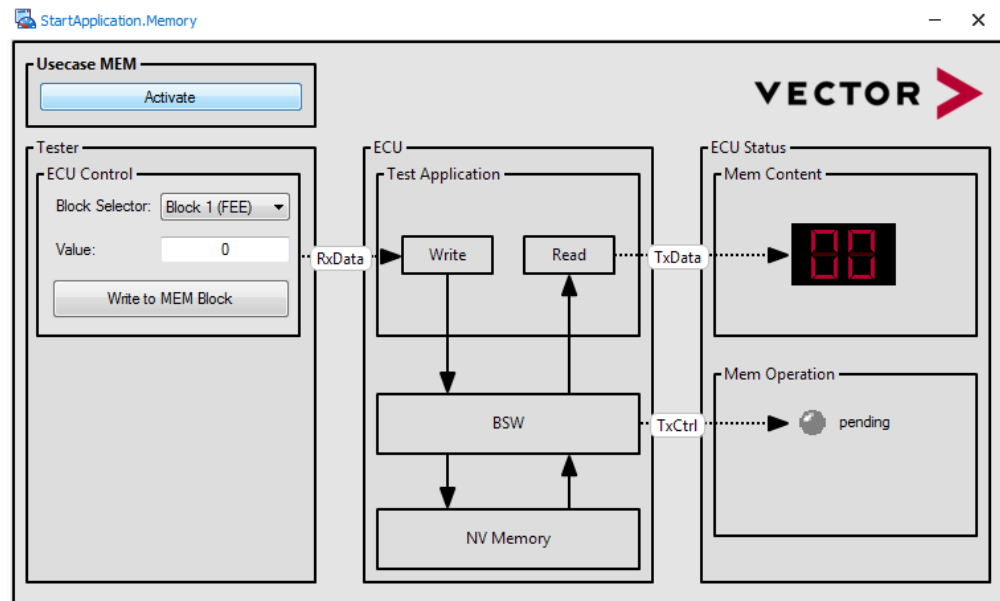
Port Prototype	Connects	Connected Port Prototype
PpNvM_RpNotifyJobEnd_StartApplication_NvMBlock1	NvM	NvM_RpNotifyJobEnd_StartApplication_NvMBlock1
PpNvM_RpNotifyJobEnd_StartApplication_NvMBlock2	NvM	NvM_RpNotifyJobEnd_StartApplication_NvMBlock2

Here the Start Application is the server for the required services.

2.11.3 CANoe

Panels

In CANoe, the application can be controlled and visualized via variable synchronization, which can be set and received with CANoe panels, as in the COM use case.

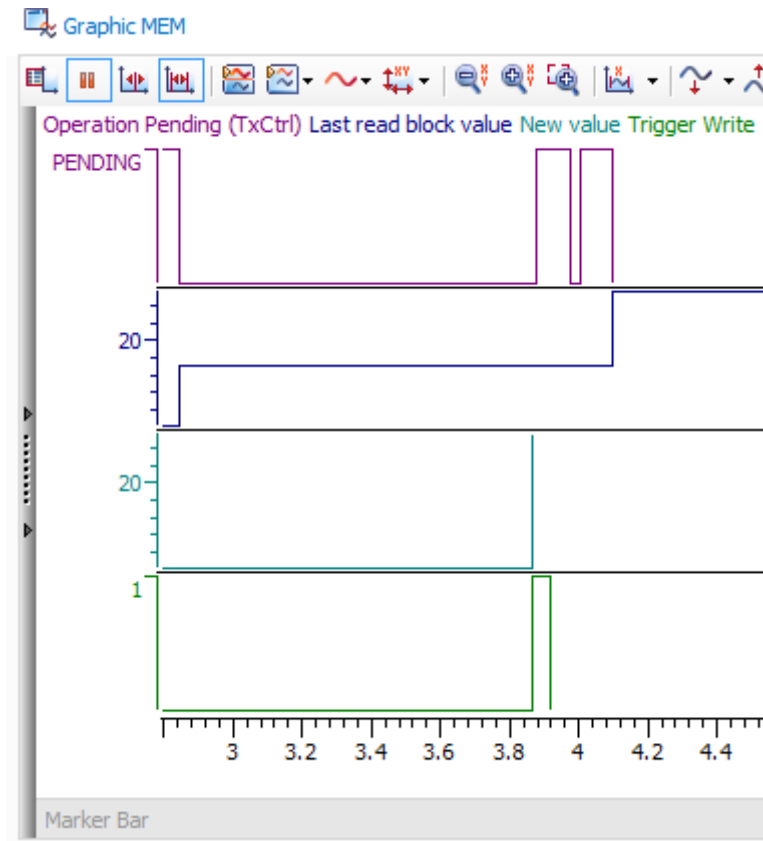


You can see the last stored value, which was read from the selected NvM Block by the cyclic runnable and sent to the TxData variable, shown in the panel memory content display or in the standard graphic window.

You can set the RxData variable to the ECU by entering a new value and press the **[Write to MEM Block]** button. If no operation is pending the value is stored in the NvM block.



Example: The pending information is sent via the TxCtrl variable. You can see if an operation is pending in the panel memory operation pending led or in the standard graphic window.



In the screen shot the last read value from the block was '14' at the beginning. At time 3,9 s a request was sent to store the value '31'. Thus, the pending flag was set in the application and sent to the TxCtrl variable. After 0.1s at time 4 s the write operation was done and the job end callback cleaned the pending flag. In the next cyclic runnable call, the read operation was started, and hence the pending flag was set and transmitted again. After the read operation was finished the pending flag was cleared at around 4.1 and the new value '31', which was read from the Nv block, was transmitted in the TxData variable.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

2.12 Use Case MemoryProtection

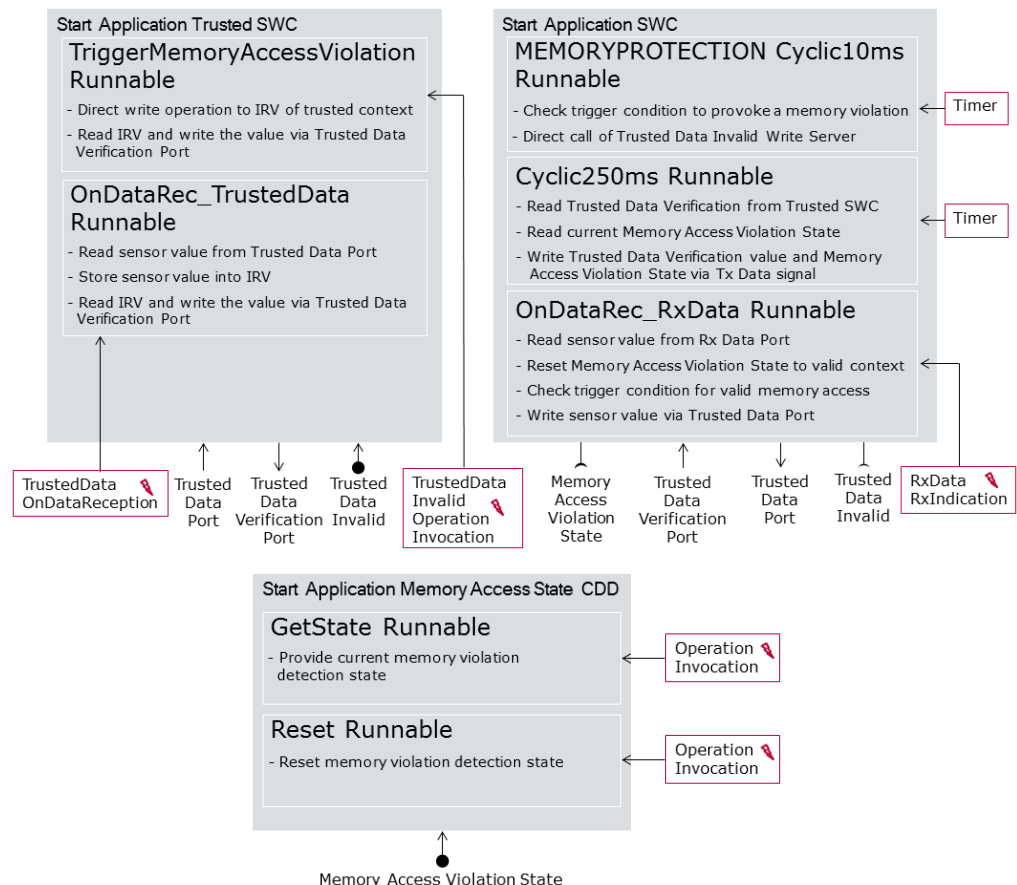
Memory Access Violation Detection

The use case MEMORYPROTECTION demonstrates the basic functionality of the Memory Protection Unit (MPU). More precisely, this is realized by writing data to a variable from a valid and invalid context and verifying that the Protection Hook is called.

The MEMORYPROTECTION use case behavior is represented by the following SWC design:

2.12.1 SWC

Structure



Behavior

The RxData reception runnable is triggered on reception of a value for RxData. Before the write access to the trusted data variable a reset of the current memory access violation state is done. Depending on the trigger condition one of the following two things will happen: In case of an invalid memory access a memory access violation will be provoked. This means that the MEMORYPROTECTION Cyclic10ms runnable will trigger a direct write operation to the IRV of the trusted context. In case of a valid memory access of the IRV the write operation will be done via S/R port.

The Cyclic250ms runnable reads the value of the IRV via the Trusted Data Verification port and the current memory access violation state from the via the Memory Access Violation State port. Both data values are transferred via the Tx Data signal to the Tester.

The two runnables of the Start Application Trusted SWC have a shared access to an IRV. The TriggerMemoryViolationAccess runnable writes the IRV without a context switch. This means the IRV is written from a non-trusted context. The MPU detects the memory access violation and prevents writing to the IRV. Afterwards the OS calls the Protection Hook. The OnDataRec_TrustedData runnable is running in a trusted

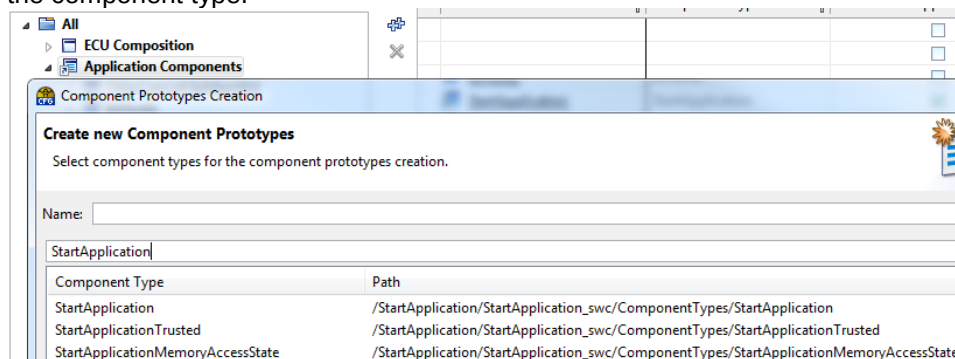
context and thus is allowed to update the IRV with the received sensor value. After the update the IRV value is send back to the Start Application SWC.

The StartApplicationMemoryAccessState CDD is used for the storage of the memory access violation state. In case the Protection Hook is called from the OS the state will be updated. The Start Application SWC is able to read and reset the current state via the server port. The CDD is only used for the evaluation and visualization of the memory access violation.

2.12.2 CFG5

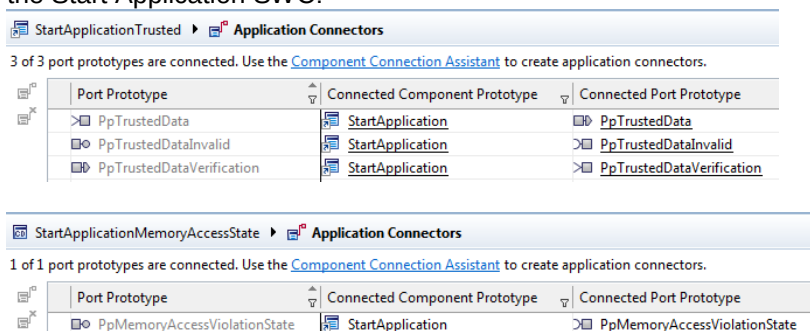
SWC

In addition to the Start Application SWC the StartApplicationTrusted SWC and the StartApplicationMemoryAccessState CDD are instantiated. This can be done in “ECU Software Components Editor” by creating new Application Component and selecting the component type.



Ports

The application connectors of the trusted SWC and the CDD need to be connected to the Start Application SWC:



Task mappings

The MEMORYPROTECTION_Cyclic10ms runnable of the Start Application SWC is mapped on a task called StartApplication_MemoryViolationTask. This task is running in the context of a non-trusted OS Application. The task identifier is used within the Protection Hook to evaluate if the memory access violation was caused by this use case:

StartApplication ▶ Task Mapping

8 of 13 function triggers are mapped. Use the [Task Mapping Assistant](#) to map the runnable entities.

Triggered Function	Trigger Category	Trigger Condition	Task
StartApplication_OnDataRec_RxData	Data Reception	PpRxData.DeSignal_Rear_...	Default_Appl_Task
StartApplication_OnDataRec_RxCtrl	Data Reception	PpRxCtrl.DeSignal_Rear_8...	Default_Appl_Task
StartApplication_MEMORYPROTECTION_Cyclic10ms	Periodical	10 ms	StartApplication_MemoryViolationTask

For the Trusted SWC the StartApplicationTrusted_OnDataRec_TrustedData runnable is mapped to a task which is running in the context of the trusted OS application:

1 of 2 function triggers are mapped. Use the [Task Mapping Assistant](#) to map the runnable entities.

Triggered Function	Trigger Category	Trigger Condition	Task
StartApplicationTrusted_OnDataRec_TrustedData	Data Reception	PpTrustedData.DeValue	StartApplication_Trusted_Task
StartApplicationTrusted_TriggerMemoryAccesViol...	Operation Invocation	PpTrustedDataInvalid.Write	

The server runnable StartApplicationTrusted_TriggerMemoryAccesViolation is not mapped to a task. This is done on purpose to provoke a memory access violation.



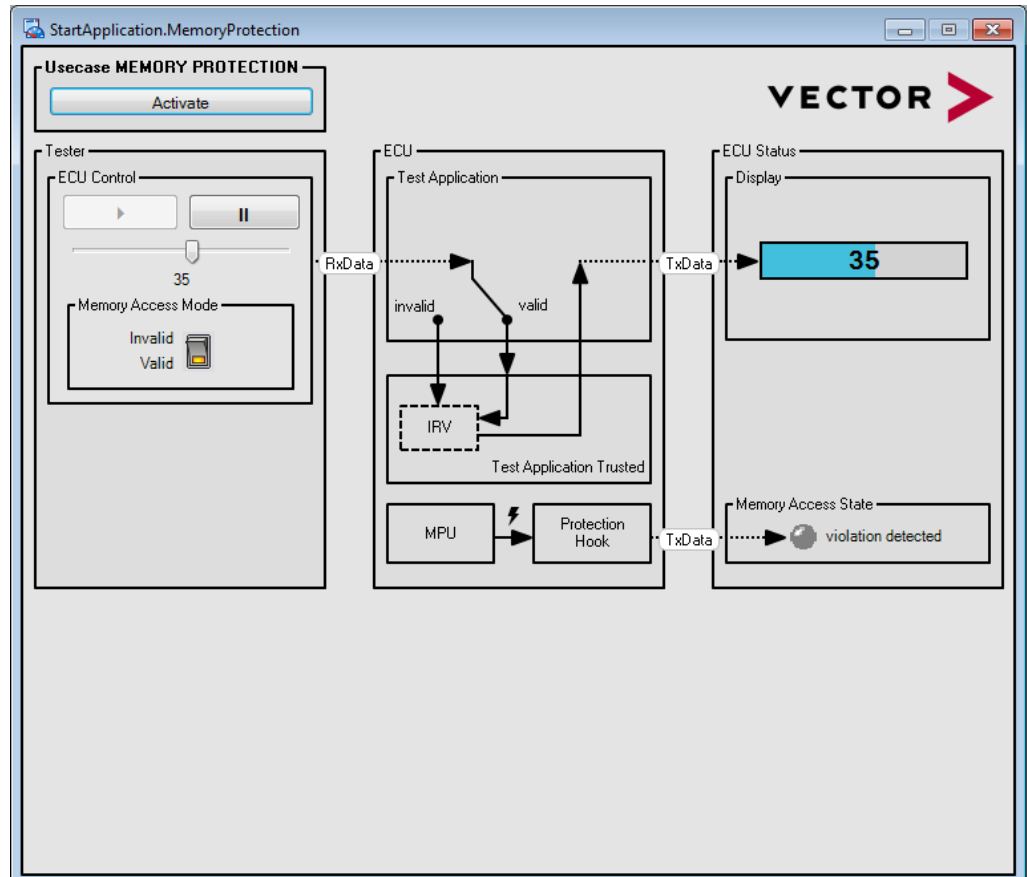
Note: The Cfg5 recognizes a possible problem and reports the following warning which is expected and correct for the Start Application MEMORYPROTECTION use case:

RTE15009	Unmapped server runnable is called over a memory protection border (1 message)
	Server runnable <StartApplicationTrusted_TriggerMemoryAccesViolation> of component <StartApplicationTrusted> is not mapped to a task.

2.12.3 CANoe

In CANoe the application can be controlled via system variables which are connected with the following CANoe panel:

Panels



The section on the left comprises the slider on the CANoe panel which generates a sensor value and the memory access mode which is send to the test application via RxData. You can press the **[Memory Access Mode]** switch to toggle between the valid and invalid memory access mode.

The test application in the middle checks the memory access mode flag and triggers a write operation to the IRV of the trusted test application. Depending on the memory access mode this will be done via a valid or invalid data path. The test application cyclically reads the value of the IRV and the current memory access violation detection state and sends those data to the tester using the TxData signal.

In case of a memory access violation the Protection Hook is called and sets the memory access violation detection flag.

On the right side the resulting values are visualized. For the valid memory access the displayed sensor value will continuously change and the memory access state LED will be switch off (gray). In case of a memory access violation the displayed sensor value will not be updated anymore and the memory access violation state LED will be switched on (yellow).

You can start the sensor value generating by pressing the **[Play]** button. The slider value increases and decreases automatically. You can also stop the value generation by pressing the **[Pause]** button and set the slider value manually.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

2.13 Use Case Multicore

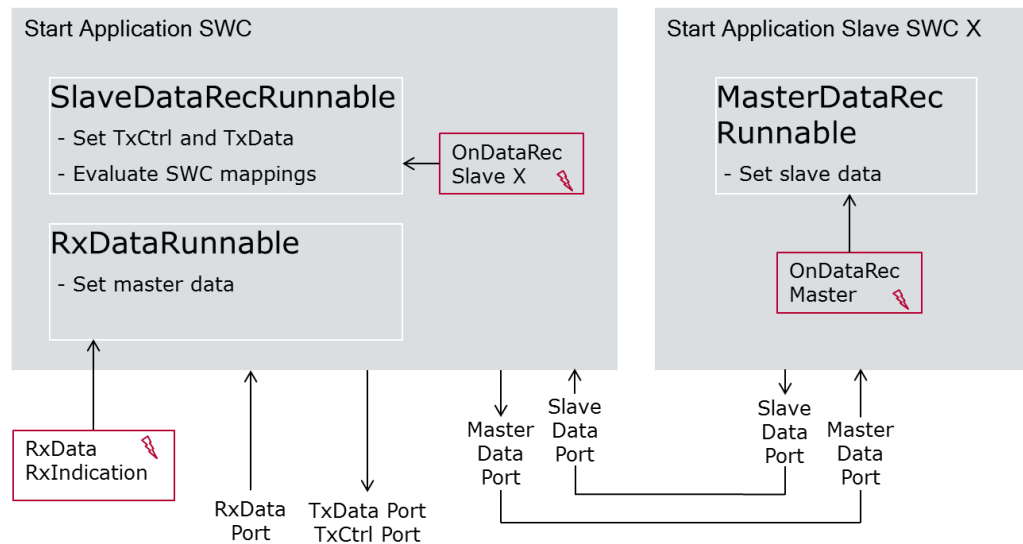
Inter core communication

The use case MULTICORE demonstrates the inter core communication between a master core and slave cores on application level. More precisely, this is realized by data exchange between the master and slave core applications via S/R ports using X-Signal communication.

The MULTICORE use case behavior is represented by the following SWC design:

2.13.1 SWC

Structure



Behavior

The RxData reception runnable is triggered on reception of a value for RxData, and it set the master data including the RxCounter and the SlaveID based on RxData value.

The MasterDataRec runnable of the slave SWC is triggered on reception of the master data, and it set the slave data, including:

- > theTxCounter value by copying the RxCounter value
- > the CoreID value by calling OS service GetCoreID().

The SlaveDataRec runnable of the master SWC is triggered on reception of the slave data, and it

- > set TxCtrl with the CoreID value
- > set TxData with the TxCounter value
- > evaluate SWC mapping by comparing the slave CoreID with the master CoreID



Expert knowledge: Evaluation of SWC mapping:

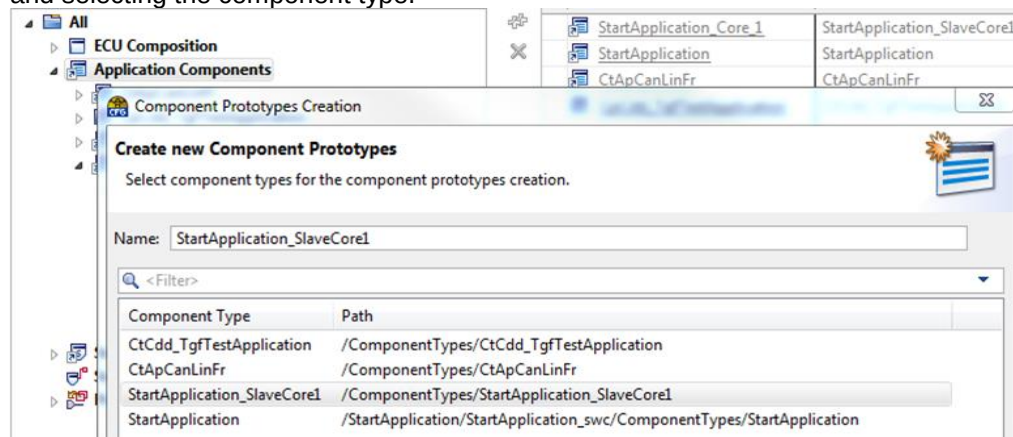
Since it only makes sense to test communication between master application and slave application when they are on different cores, evaluation of SWC to core mapping is run for every CoreID sent from slave.

Master application compares its CoreID with the CoreID sent from slave, and returns mapping error when they are equal.

2.13.2 CFG5

SWC

For each tested AUTOSAR core, the associated SWC has to be instantiated. This can be done in “ECU Software Component” by creating new Application Component and selecting the component type.



Ports

For each slave core, Send/Receive ports have to be connected to master core SWC, which can be done in the “ECU Software Components” comfort view:

Port Prototype	Connected Component Prototype	Connected Port Prototype	Port Interface Mapping
PpSlaveCoreData	StartApplication	PpSlaveCoreData1	
PpMasterCoreData	StartApplication	PpMasterCoreData1	

Task mappings

Task mappings are required for all runnables on master and slave applications which are triggered on data reception, which can be done in the “ECU Software components” comfort view:

Triggered Function	Trigger Category	Trigger Condition	Owner	Task
StartApplication_OnDataRec_SlaveCore2	Data Reception	PpSlaveCoreData2.DeInterCoreSlave	Component StartApplication	Default_Appl_Task
StartApplication_OnDataRec_SlaveCore1	Data Reception	PpSlaveCoreData1.DeInterCoreSlave	Component StartApplication	Default_Appl_Task
Triggered Function	Trigger Category	Trigger Condition	Owner	Task
StartApplication_OnDataRec_MasterCore	Data Reception	PpMasterCoreData.DeInterCoreMaster	Component StartApplicationSlaveCore1	Default_Appl_Task



Expert knowledge: Unique SWC to core mapping

Generally the mapping chain is as follows:

runnables → tasks → OsApplications → cores.

The runnables are mapped to the tasks, and the tasks are mapped to the OsApplications, which are on different cores.

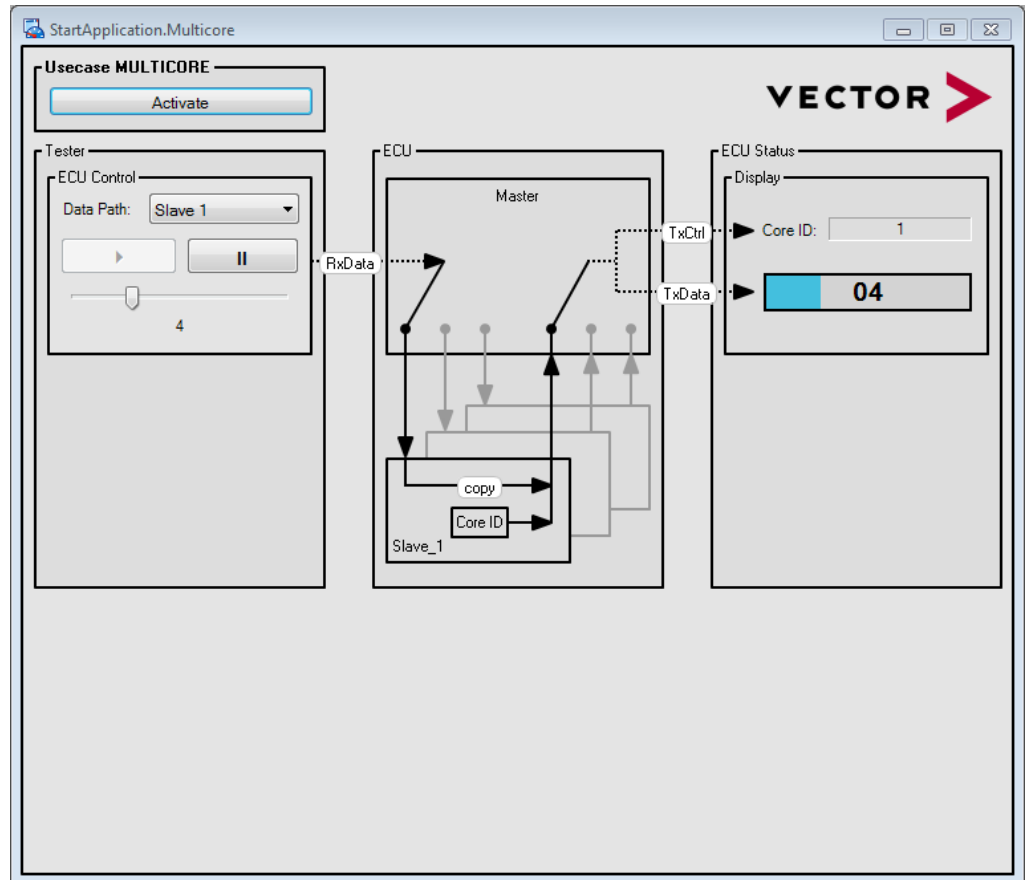
RTE error occurs if the runnables of the same SWC are mapped to the tasks bound to different cores.

This mapping guarantees a unique SWC to core mapping.

2.13.3 CANoe

In CANoe the application can be controlled via system variables which are connected with the following CANoe panel:

Panels



The section on the left comprises the slider on the CANoe panel which generates a sensor value and the Data Path combo box which sets the slave id. Both values are transferred to the master application via RxData.

The master application in the ECU sends the sensor value to corresponding slave application, which triggered the setting of slave core data. The slave application copies the sensor value and sends it with its own CoreId back to the master application.

The master application set the TxData value with the CoreId value, which is used for evaluating the SWC mapping. It also set the TxCtrl value with the sensor value sent from the slave application, which is shown on the panel as the progress bar on the right display section.

In summary, this results in the following data path:

ECU control → master application → slave application → master application → Display

You can start the sensor value generating by pressing the **[Play]** button. The slider value increases and decreases automatically. You can also stop the value generation by pressing the **[Pause]** button and set the slider value manually.

If you have more than one slave applications, you can switch between different slave applications by choosing different data path using the Data Path combo box.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

2.14 Use Case NM

Requesting FullCom and NoCom (ComM)

The use case NM demonstrates how the application can control and check the communication channel status via ComM users.

NM Protocol (Optional)

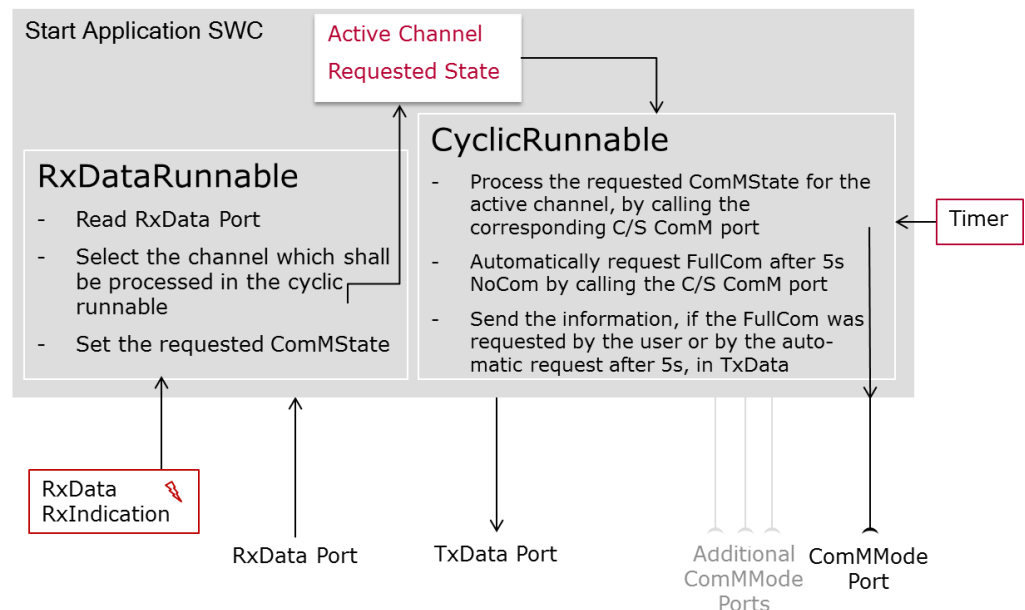
Additionally, the network management protocol handling can be demonstrated for channels with Autosar NM. More specifically, the communication channel can be kept active by another node although the application of the ECU has released its communication request.

Remote wakeup (Optional)

The use case can also demonstrate that a communication channel in sleep state can be started by another node via remote wakeup. Note: this requires that the ECU can detect wakeup events for the channel.

2.14.1 SWC

Structure



Behavior

The RxData reception runnable is triggered on reception of a value for RxData. The RxData value is evaluated as follows:

- The most significant bit denotes whether a FullCom or NoCom request shall be issued.
- For an ECU connected to multiple communication channels, the other bits determine the channel for which the request shall be issued, i.e. the active channel.

The cyclic runnable performs the following actions for the channel selected by the RxData value:

- When the RxData signal indicates that FullCom shall be requested for a channel, FullCom is requested for the corresponding ComM user.
- When the RxData signal indicates that NoCom shall be requested for a channel and the channel is in FullCom, NoCom is requested for the corresponding ComM user.
- In case the channel is in NoCom for more than ca. 5s, FullCom is requested for the corresponding ComM user and released after another 5s. The TxCtrl value denotes whether this timer controlled FullCom request is active.

2.14.2 CFG5

The ComM module offers a service port for each ComM user, in order to request communication for this user or to get the current communication state for this user. They have to be connected in the “ECU Software Components” comfort view:

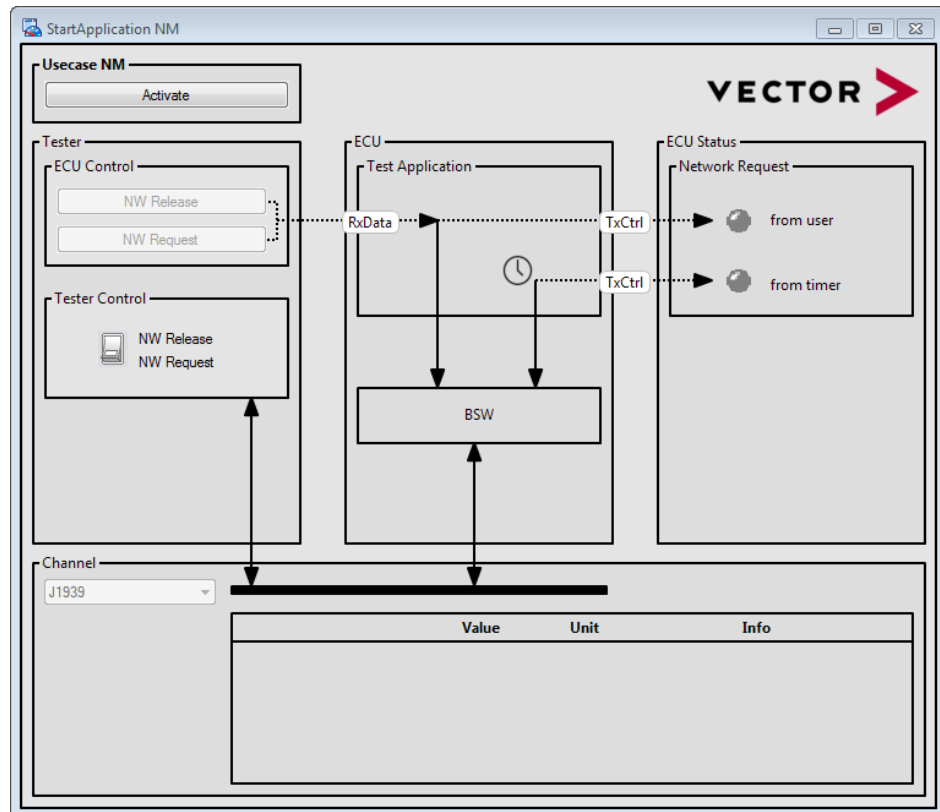
Port Prototype	Connected Component Prototype	Connected Port Prototype
UR_USR_FlexrayCluster_1_f26020e5	ComM	UR_USR_FlexrayCluster_1_f26020e5
UR_USR_CHNL_f276589c	ComM	UR_USR_CHNL_f276589c
UR_USR_CHNL_8571680a	ComM	UR_USR_CHNL_8571680a

Here the Start Application is the client of this provided service.

2.14.3 CANoe

In CANoe, the application can be controlled with the following panel.

Panels



The panel consists of the following sections:

The section on the left comprises the buttons for controlling the network request both for tester and ECU side:

- > The buttons **[NW Request]** and **[NW Release]** can be used to control the status of the FullCom request on ECU side. When one of these buttons is pressed, the RxData values changes. Note: The RxData value is synchronized between tester and ECU using one of the communication channels. In case this channel is currently in NoCom mode, pressing buttons will have no immediate effect. This is indicated in the panel by a red **[X]** on the RxData path.
- > The switch **[Tester]** can be used to control the status of the FullCom request on Tester side. For a channel with Autosar NM this switch controls whether the Tester transmit NM messages for keeping the channel active.

The middle section depicts the behavior of test application (see section 2.14.1)

The section on the right displays status information from the ECU which is transferred via TxCtrl:

- > The LED labelled “from user” denotes the status of the FullCom request which is controlled via RxData. You set can the status via the “NW Request” and “NW Release” buttons on the left.
- > The LED labelled “from timer” denotes the status of the timer controlled FullCom request.

The bottom section provides a switch to change the communication channel. Additionally, status information for the currently selected channel is displayed.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

2.15 Use Case SCC

SCC Callback Interface

The use case Smart Charging Communication (SCC) demonstrates the basic communication between an Electric Vehicle (EV) and an Electric Vehicle Supply Equipment (EVSE) on application level. More precisely, it demonstrates the SCC callback handling on application level.

SCC Add-On

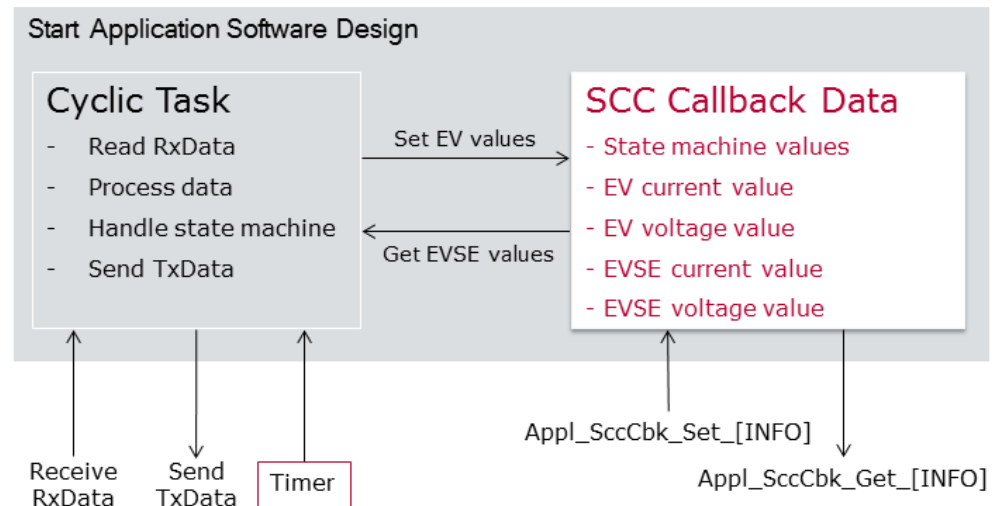
The use case requires the installation of the Smart Charging Communication Package add-on which can be downloaded on the Vector homepage. Therefore visit <https://vector.com> > Downloads > select 'Electric Vehicles' within the 'Products and Topics' category. Afterwards select 'Add-Ons/Freeware' within the 'Categories' tab. Finally click on the 'show result' button and download the package as 32bit or 64bit version according to your requirements.

SCC_ChargePoint.dll

The SCC_ChargePoint.dll provided by the add-on is able to simulate the behavior of an EVSE and will be the counterpart of the EV. For the use case it is required that the DLL is added as component to the CAPL Node which references the CanoePanelAbstraction.can. Additional information can be found in the ReadMe.txt (CANoe\StartApplication\ReadMe.txt).

2.15.1 Software Design

Structure



Behavior

The Cyclic Task on the left side is triggered each 250ms. In the context of the task the RxData value is read and processed. The RxData value contains the information about the connect state and in addition the EV current value and the EV voltage value. Those values are stored within separate variables for later processing within the state machine and the SCC callback handling.

The state machine handles the SCC Protocol flow. It switches between the different states like "Session Setup", "Service Discovery", "Charge Loop" etc. based on several state machine variables.

The data exchange between the SCC Application and the SCC BSW module is realized via the SCC callback interface which is accessing the data on the right side. Setting or getting data is always initialized by the SCC BSW module. The [INFO] label at the end of the Appl_SccCbK_Set and Appl_SccCbK_Get is a placeholder for the various callbacks which needs to be implemented. There are different callbacks for the ISO/DIN standard combined with the AC/DC charge mode for example:

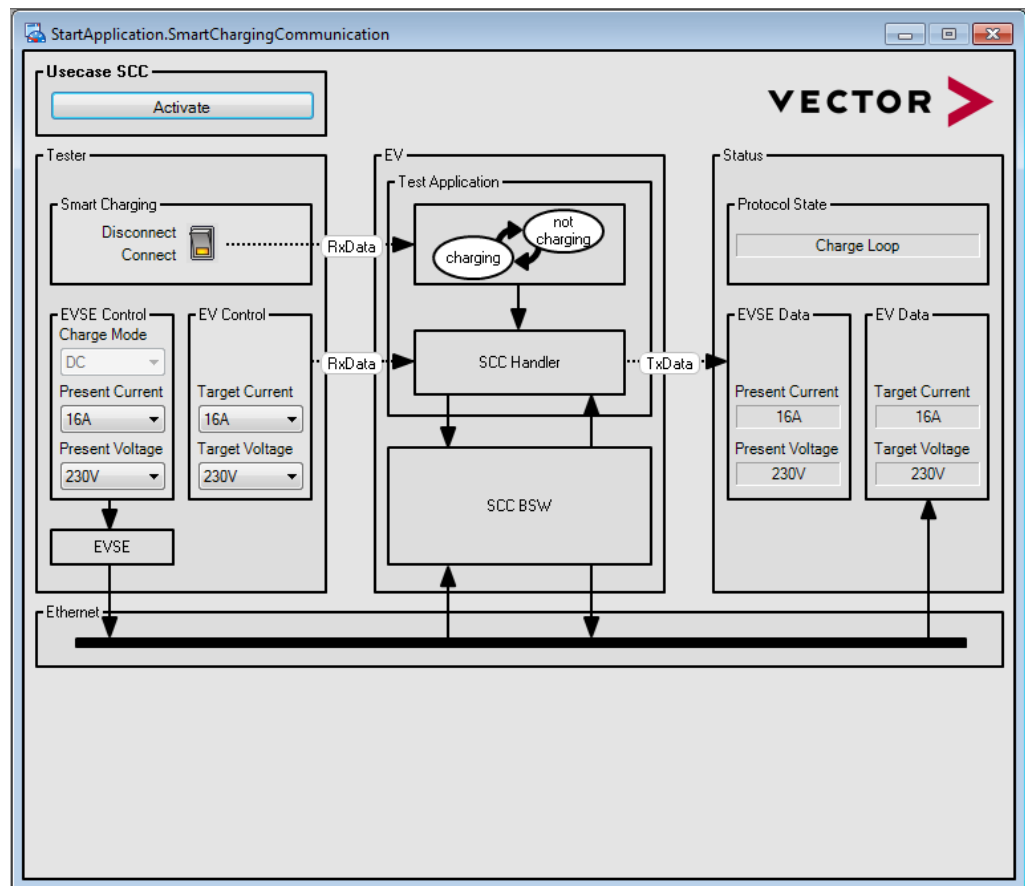
- > Appl_SccCbK_Get_ISO_AC_EVMaxVoltage
- > Appl_SccCbK_Set_DIN_EVSEMPresentCurrent

At the end of a processing cycle the Cyclic Task get the EVSE values from the SCC callback data and update the TxData value which is send out periodically.

2.15.2 CANoe

Panels

In CANoe the Smart Charging Communication can be parameterized by synchronizing the control values with system variables. The system variables are connected in this use case to the following panel:



On the Tester side which is represented by the left box, the Electric Vehicle (EV) and the Electric Vehicle Supply Equipment (EVSE) can be parameterized. Based on the EVSE Control section the preferred **[Charge Mode]** can be set to AC or DC. The selection can be restricted to one of both modes depending on the supported configuration of the SCC BSW module. When DC mode is selected:

- > the **[Present Current]** / **[Present Voltage]** of the EVSE can be configured
- > the **[Target Current]** / **[Target Voltage]** of the EV can be configured

When AC mode is selected:

- > the **[Maximum Current]** / **[Nominal Voltage]** of the EVSE can be configured
- > the **[Max Current]** / **[Max Voltage]** of the EV can be configured

The **[Disconnect]** / **[Connect]** switch trigger the SCC Protocol handling. After connecting the EV to the EVSE some of the controls can be disabled. Main reason is that those parameters could not be modified while the protocol is handled and the application is in charging state. To enable the controls again the EV needs to be disconnected.

The SCC application part is visualized in the middle section. The application contains a state machine which is represented by the “charging” / “not charging” box. Depending on the current state within the SCC protocol the SCC BSW is using the callback interface to get or set different data. In this context the SCC BSW is for example requesting the maximum current and nominal voltage of the EVSE in the AC charge mode. On the other side it is providing data for the EVSE like the maximum current and maximum voltage of the EV.

On the left side the current processing state and the related data are summarized within the Status box. The current protocol state of the application state machine can be found on top. Below the protocol state the exchanged data of the EVSE and EV are shown.

The EVSE control values are used to parameterize the SCC_ChargePoint.dll. The resulting data flow within the application context will be: EVSE receives a request from the EV → the request information will be provided via Ethernet to the EV → SCC BSW → SCC Callback Interface → SCC Application → Status display.

The other way round the EV control values are stored within the SCC Application → requested by the SCC Callback Interface → SCC BSW → Ethernet → EVSE which is represented by the SCC_ChargePoint.dll → Status display.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

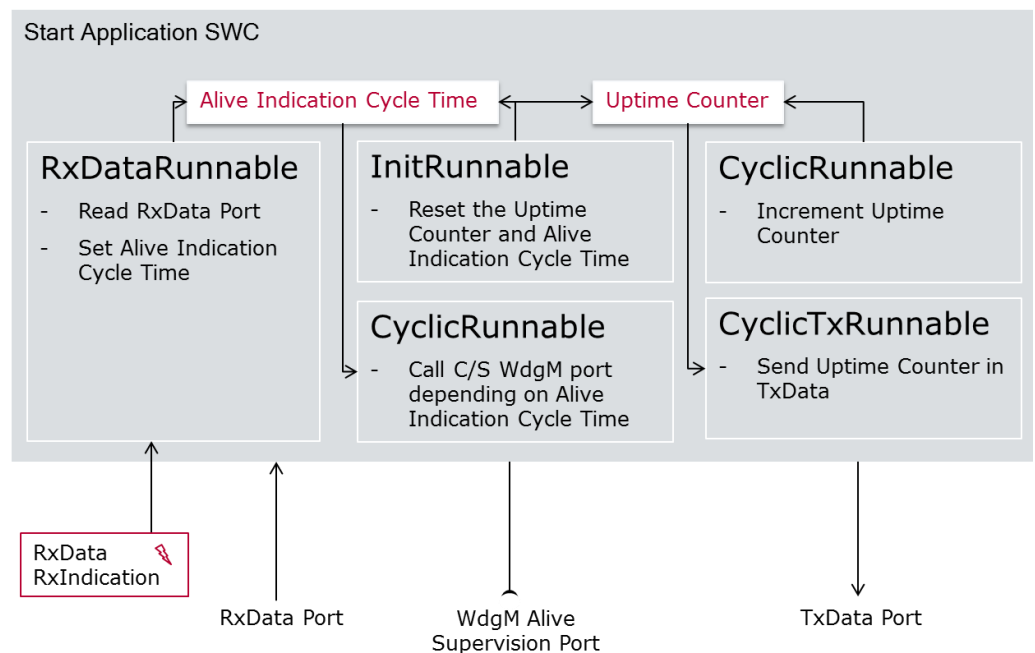
2.16 Use Case WDG

Alive Supervision

The use case WDG demonstrates basic alive supervision handling for a supervised entity of the WdgM. The WdgM triggers a reset of the ECU if the operation CheckpointReached of the WdgM AliveSupervision service port is not called often enough in the time period configured for the supervised entity.

2.16.1 SWC

Structure



Behavior

The RxData reception runnable is triggered on reception of a value for RxData:

- > The value sets the Alive Indication Cycle Time with which the CheckpointReached operation of the WdgM AliveSupervision port is called.

The init runnable performs the following actions:

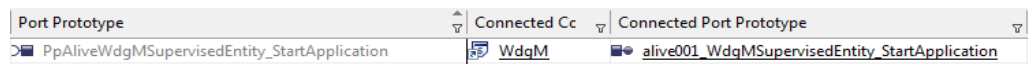
- > Reset the Uptime Counter to 0.
- > Reset the Alive Indication Cycle Time to its initial value.

Different cyclic runnables perform the following actions:

- > Increment the Uptime Counter every second. If the uptime is greater than 255 it will not be incremented anymore.
- > Send the current Uptime Counter using the TxData signal every 250ms.
- > Call the CheckpointReached operation of the WdgM AliveSupervision port cyclicly depending on the current Alive Indication Cycle Time.

2.16.2 CFG5

The WdgM module offers a service port for each supervised entity. This port contains an operation to indicate to the WdgM that a Checkpoint has been reached. It has to be connected in the “ECU Software Components” comfort view:

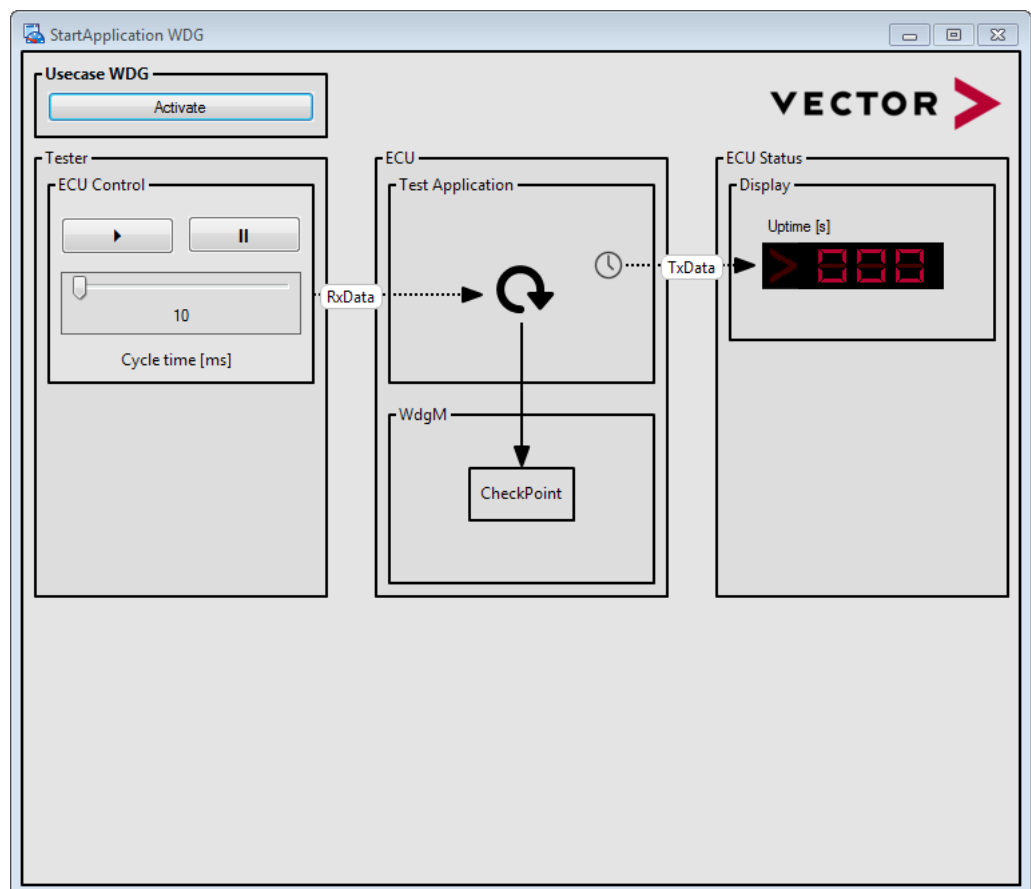


Here the Start Application is the client of this provided service.

2.16.3 CANoe

In CANoe, the application can be controlled with the following panel.

Panels



The panel consists of the following sections:

The section on the left comprises the buttons for controlling the Alive Indication Cycle Time which is transferred via RxData:

- > The slider can be used to manually increase or decrease the cycle time.
- > The buttons **[Play]** and **[Pause]** can be used to activate or deactivate the automatic incrementing of the cycle time.

The middle section depicts the behavior of the test application (see section 2.16.1).

The section on the right displays status information from the ECU which is transferred via TxData:

- > The display shows the current uptime of the ECU. If the uptime is greater than 255 seconds a greater-than sign is shown in front of the uptime and the uptime itself stays at 255 seconds.

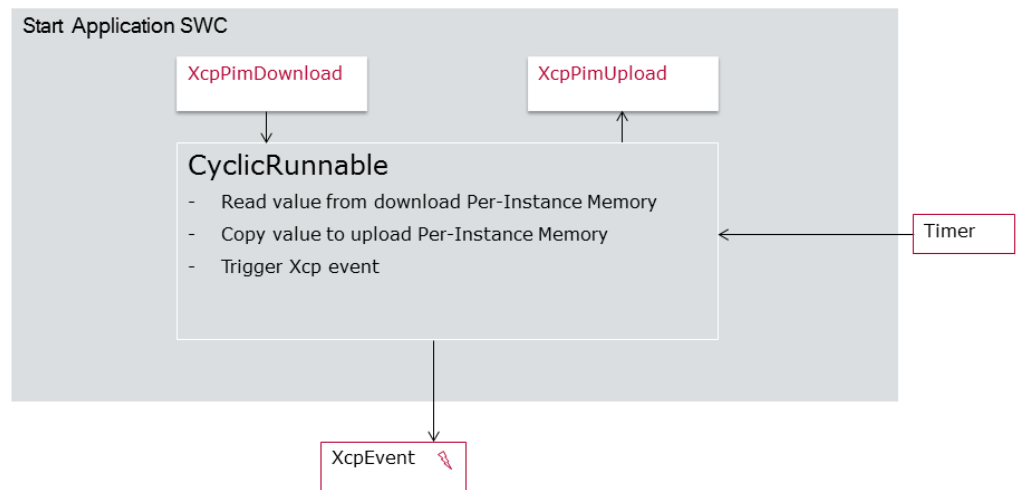
Use case activation The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.

2.17 Use Case XCP

Calibration support of Per-Instance Memories The use case XCP demonstrates the basic download and upload of data via the Universal Calibration Protocol. More precisely, it demonstrates the interaction of a Master and a Slave using Per-Instance Memories with calibration and measurement access. This behavior is represented by the following SWC design:

2.17.1 SWC

Structure



Behavior

The cyclic runnable is triggered each 250ms. Within this runnable, the data of the Per-Instance Memory which is written via XCP download is read. The read value is copied into the Per-Instance Memory which is uploaded via XCP. Afterwards the XCP event is triggered.

If you have a multi-bus ECU only one pair of the Per-Instance Memories for the download and upload direction is generated. This pair will be used for all given busses which have a supported XCP configuration available.

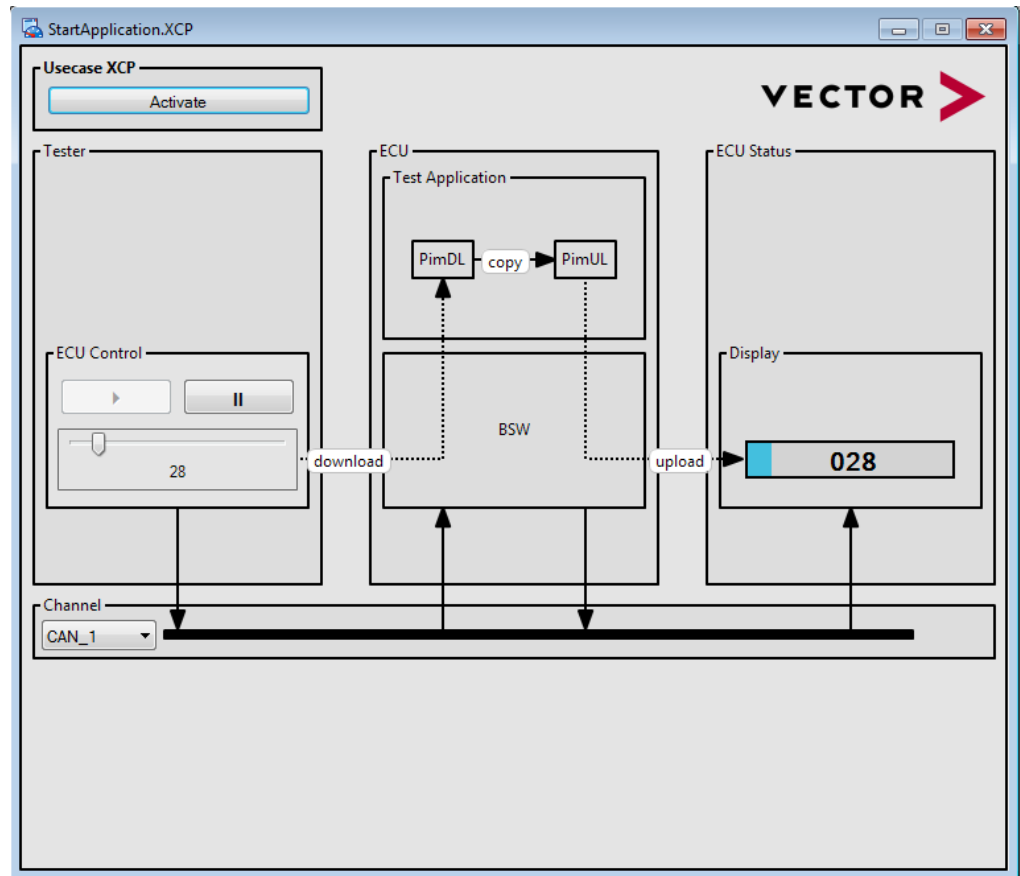


Expert knowledge: An important part of the A2L file is the location of the variables in the memory of the Slave. Hence to ensure a reliable interaction between the Master and the Slave the A2L file needs to be updated after each software build.

2.17.2 CANoe

Panels

CANoe acts as the Master and plays the counterpart of the ECU which represents the Slave. CANoe allows you to see and to influence the data element values, by synchronizing them with its system variables, as described in the previous chapter. The system variables are connected in this use case to the following panel:



The slider in the CANoe panel generates a calibration value and writes this value to the RAM address of the download Per-Instance Memory. The application in the ECU cyclically reads this value and writes it into the corresponding upload Per-Instance Memory. The upload is handled by the Master and the value is finally displayed in the progress bar in the panel. It should be equal to the calibration value. In summary, this results in the following data path: ECU Control → XCP download → ECU → XCP upload → Display.

You can start the calibration value generation by pressing the **[Play]** button. The slider value increases and decreases automatically. You can also stop the value generation by pressing the **[Pause]** button and set the slider value manually.

If you have more than one channel with a supported XCP configuration, you can choose your communication path using the channel select box.

Use case activation

The **[Activate]** Button is relevant if you have more than one use case available, because some SWC ports are used in the implementation with different semantic in different use cases (see 2.18). This button needs to be pressed before using the other control elements for this use case.



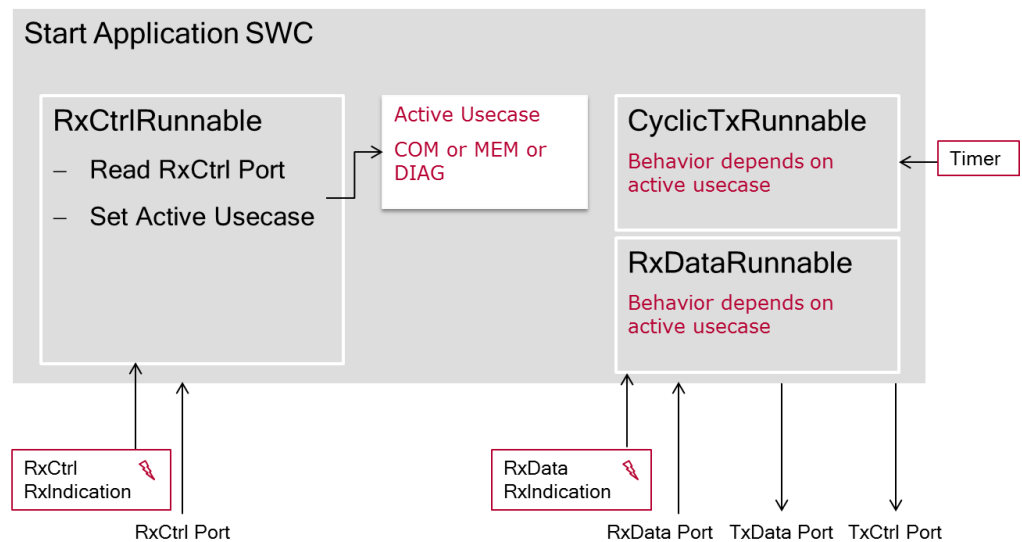
Expert knowledge: The XCP use case panel utilizes the CANoe build-in XCP infrastructure for configuration and communication handling.

2.18 Multiplexing of Variables

Sender/Receiver on runnable level

All use cases need a way to send data to the application from CANoe as well as a way to receive data from the application. This is done by the synchronization of variables as described in section 2.1.

In order to use as few variables as possible and to re-use the data elements within the SWC throughout the different use cases, one data element (RxCtrl variable) is used to switch the semantic of the received data.



Depending on the active use case, the meaning of e.g. the RxData changes. For COM RxData switches the value pattern, for MEM it contains the value to be stored and for DIAG it determines if the DEM event should be set to pass or failed.

Each time you press the **[ACTIVATE]** Button on a use case specific panel, a certain value is sent to the RxCtrl variable so that the implementation switches to the use case specific behavior. Thus the semantic of the other data ports changes, although the same variables are used, with their same representation on the communication bus.

Hence only four variables are enough to handle the data exchange between CANoe panels and the ECU.

Use Case	RxData	TxCtrl	TxData
COM	Scaled sensor value of one certain channel. Based on this value the TxData value is calculated. Other channels use additional signals → see panel	Unused	Transfers the last received sensor value of one certain channel. Other channels use additional signals → see panel
DIAG	Depending on the received value the DTC is set to failed (1) or passed (0) with the "SetEventStatus" operation of the DiagnosticMonitor Port. Additionally the event counter can be reset (2). The reset functionality is only for testing reasons and not visible in the panel.	Unused	Transfers the last event counter value. This functionality is only for testing reasons and not visible in the panel. Further the value range is limited to 0...255.
ETH	Unused	Unused	Unused
FBL	Unused	Unused	Unused
J1939DIAG	Depending on the received value the DTC is set to failed (1) or passed (0) with the "SetEventStatus" operation of the DiagnosticMonitor Port.	Unused	Unused
MEM	Contains following information: - Block which is active - If block shall be written - Which value shall be written to the active block (0..15) Value is only written to NvM block if it is not pending.	Contains following information: - Block which is active Status information for the corresponding block (e.g. pending)	Contains following information: - Transfers the last value read from the active NV block - Block which is active
MEMORY PROTECTION	The sensor value and the memory access mode which is send to the test application. The sensor value will be written to the IRV of the trusted test application.	Unused	Transfer the last read IRV value and the current memory access violation state value.
MULTICORE	The sensor value which is forwarded from the master core to the slave core.	The Core Id of the slave core which is transferred to evaluate the SWC mapping.	Transfers the last received sensor value of the selected slave core.
NM	The most significant bit denotes whether a FullCom or NoCom request shall be issued. For an ECU connected to multiple communication channels, the other bits determine the channel for which the request shall be issued.	Indicates whether a timer controlled FullCom request or a user controlled FullCom request is active.	Unused

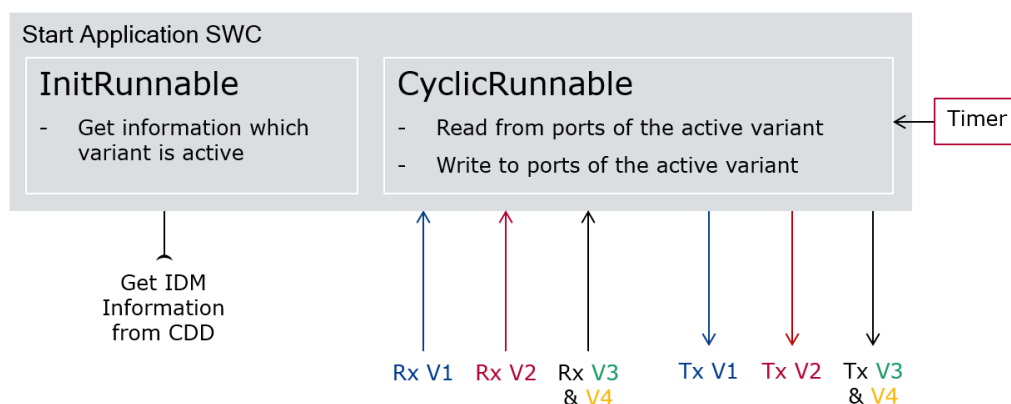
SCC	Transfers the current and voltage values of the EV. In addition it contains the connection state connected (1) and disconnected (0).	Unused	Transfers the current and voltage values of the EVSE.
WDG	The cycle time in milliseconds with which the CheckpointReached operation of the AliveSupervision port is called.	Unused	Transfers the current uptime.
XCP	Unused	Unused	Unused

2.19 Using Start Application in an IDM Configuration

- IDM use case** The following section is only relevant if this delivery is an IDM configuration. For this delivery following variants are supported by the SWC application:
- Supported variants** Invariant configuration
- The SWC application supports the COM use case for all those variants.
- Default variant** The default (main) variant is 'Invariant configuration'.
- This variant is the default active variant when the ECU is started and can contain additional use cases.
- All other variants of the BSW configuration are not considered by the SWC application.

2.19.1 SWC

Structure



Behavior

The SWC has sender/receiver ports for each supported variant. Sometimes multiple variants can share a port if the mapped signal is available in all those variants.

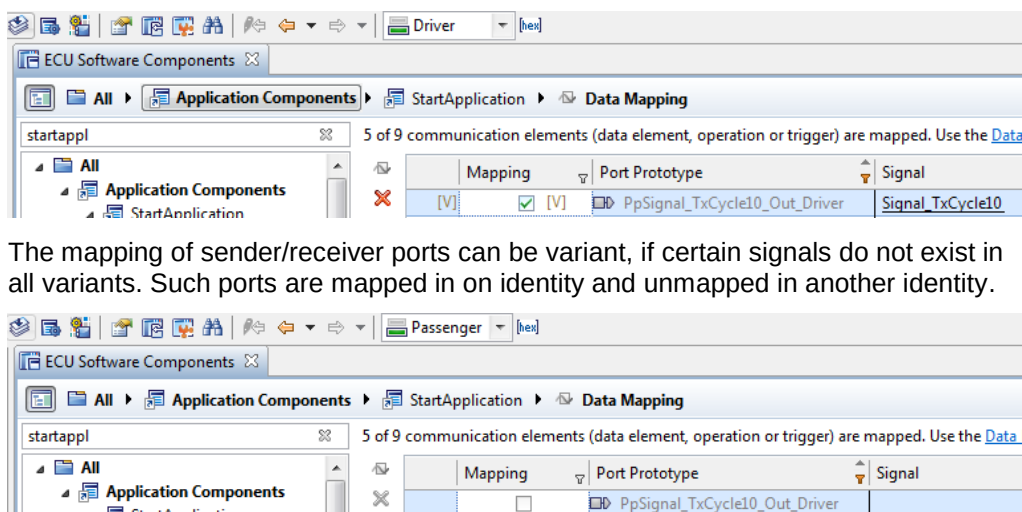
CDD

Additionally, the SWC has a client port to a CDD which tells the SWC which variant is active in the BSW. Dependent on that information the SWC application can read/write the ports of the currently active variant. The CDD (StartApplicationIdentityManagerInformation) itself gets its variant information during the ECUM callout 'EcuM_DeterminePbConfiguration'.

The CDD is only necessary at all if more than one variant is considered.

2.19.2 CFG5

Variant mapping



The mapping of sender/receiver ports can be variant, if certain signals do not exist in all variants. Such ports are mapped in one identity and unmapped in another identity.

2.19.3 Code

ECUM Callout

The BSW is capable of running in each configured variant. Which variant shall be active, has to be determined in the user callout function 'EcuM_DeterminePbConfiguration' in the source file 'EcuM_Callout_Stubs.c'.

The Start Application provides a sample implementation for this callout function, which tells the BSW which variant shall be active and stores this information in a CDD (StartApplicationIdentityManagerInformation), so it can be accessed by the SWC application.

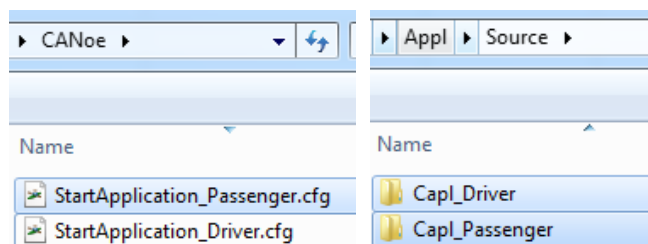
To run your ECU in a different variant, please refer to the comments of this implementation part.

2.19.4 CANoe

Dependent on which variant is active on ECU side, its communication behaviour might change:

- > Variants might have different messages on the same channel
- > Variants might have the same message (same ID) on the same channel with different layout
- > One variant might have more or other channels than the other variant

Thus in many cases one CANoe test setup cannot play the counterpart to all possible variants of an ECU. In order to avoid reconfiguration of one common test setup, there are distinct test setups for each supported variant. Those are named 'StartApplication_*[Variant name]*'. The related files are located in variant specific sub folders:



Note: Please ensure to load the right test setup according to the chosen variant on ECU side. Otherwise the Start Application use cases might not work as expected.

To validate that tester side and ECU side are consistent, execute the test script and check the report:

2. Verify that the active ECU variant is equal to the expected ECU variant of this CANoe configuration: Passed		
11.561703	Resume	Immediately resumed on setup of wait condition.
	reason	
11.561703	1	Waited for occurrence of any of 1 value condition(s).
11.561703		System variable 'StartApplication::IdmActiveVariant': = 1 (Passenger), condition: == 1 (Passenger)

This check is overdue and not executed if only one variant is considered.

2.20 Using Start Application without RTE

Calls to ASR ports replaced by BSW API calls

Given this delivery does not contain a RTE, the Start Application is not modelled as SWC and thus cannot call ASR ports. In this case Sender/Receiver ports and client ports are replaced by calls to the corresponding public BSW APIs, for example:

- > `RteWrite_Signal_X(Value) → ComSendSignal(Signal_X, Value)`
- > `Rte_Call_UR_USR_User_RequestComMode(ComMode) → ComM_RequestComMode(User, ComMode)`

In order to keep the Start Application implementation readable, the implementation calls the port API as it is done with RTE. As the port API is not available, it is defined as function like macro within StartApplication.c:

```
#define Rte_Read_PpRxCtrl_DeSignal_Rear_8bit_In(data) Com_ReceiveSignal( ComConf_ComSignal_Signal_Rear_8bit_In, data )
#define Rte_Read_PpRxData_DeSignal_Rear_16bit_In(data) Com_ReceiveSignal( ComConf_ComSignal_Signal_Rear_16bit_In, data )
```

ASR server ports replaced by callback functions

Server ports provided by the Start Application are replaced by callback functions. Those callback functions are configured at the corresponding ECUC element, for example:

- > `NvMNotifyJobFinished` port (see section 2.11.1) → A callback function is set at the corresponding NV block and implemented by the Start Application

All callback declarations can be found in the file StartApplication.h.

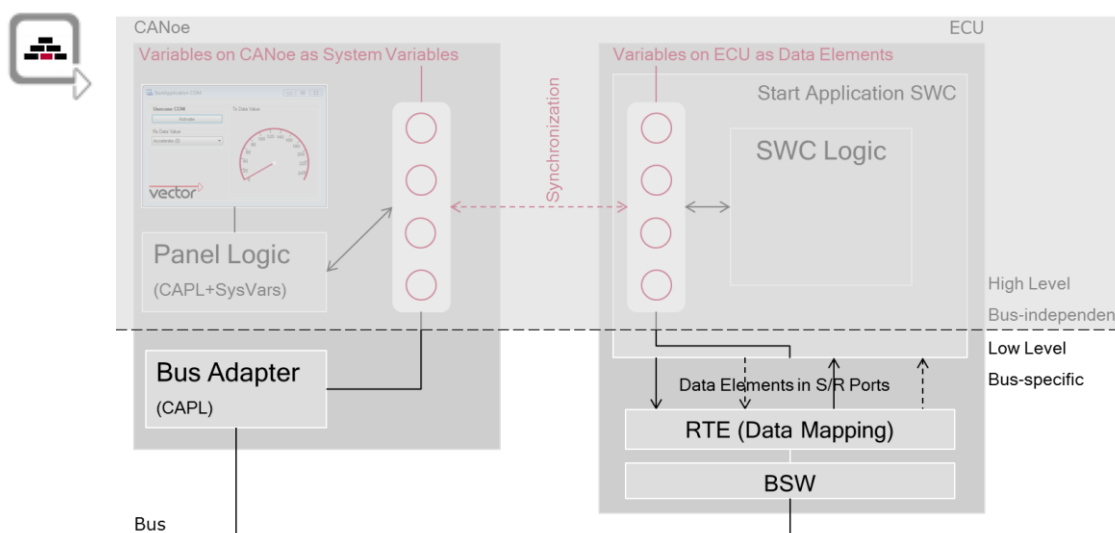
All callback implementation can be found in the file StartApplication.c.

Task implementation

Without the SWC concept there are no runnables available, which can be mapped to tasks. Therefore, the runnable functions are declared in StartApplication.h. The task implementations in the file BswInit.c call those functions.

3 Bus Specific Functionality – Low Level

This chapter guides you through the low level part, i.e. the mapping of the synchronized variables to specific elements like COM Signals, COM Signalgroups, LdCom PDUs, etc. Those elements have to be transmitted on a bus like CAN, LIN, FlexRay, etc. This chapter shows the mapping on CANoe (Bus Adapter) side as well as on ECU side (Data Mapping).



3.1 ECU Data Mapping

Communication

The data elements, which represent the variables in the SWC application, must be contained in ports in order to use them for inter ECU communication. Either both data elements of the same direction can be contained in one S/R port (e.g. for a COM SignalGroup or an LdCom Pdu) or in two distinct S/R ports (e.g. for COM Signals). That means the SWC's port is already constructed in a way that it fits to the element to which it shall be mapped to.

The screenshot shows an exemplary mapping to COM Signals. In this case there are four ports with one data element each, mapped to COM Signals.

Data Mapping

4 of 4 data elements / operations are mapped. Use the [Data Mapping Assistant](#) to map the data elements / operations to signals.

Mapping	Port Prototype	Data Element / Operation	Direction	Channel	PDU	Signal
☒	PpTxCtrl	DesT0189D_A_b00r01	Tx	Flexray_1	p00T0189D_A_b00r01	sT0189D_A_b00r01
☒	PpTxData	DeComFrSig_8Bit_5	Tx	Flexray_1	ComFr_3_5	ComFrSig_8Bit_5
☒	PpRxCtrl	Desig02Rx_0x770_IL	Rx	CHNL	msgRx_0x770_IL_oCAN00	sig02Rx_0x770_IL
☒	PpRxData	Desig03Rx_0x229_IL	Rx	CHNL	msgRx_0x229_IL_oCAN00	sig03Rx_0x229_IL

Such mappings can be done within CFG5 in the “ECU Software Components” comfort view. The further mapping from such COM signals to bus frames is contained in the BSW configuration and derived from your specific input data for communication (EcuExtract / Fibex / Dbc).

Diagnostic

The diagnostic communication is also contained in the BSW and derived from your specific input data for diagnostics (ODX/EcuExtract).

3.2 CANoe Data Mapping

Communication

In order to send new variable values via inter ECU communication, the CANoe panels need to put their value in elements like COM Signals etc. and additionally trigger a transmission of the frame which contains this element.

In order to keep the panel bus-independent, the bus-specific part is handled in a CAPL function:

Panel control triggered → System variable changes → System variable handler activated → bus-specific sent function triggered

Since CANoe can access your specific input data for communication (EcuExtract / Fibex / Dbc), this can be done in CAPL functions using symbolic name values. It is important that the same mapping is made in CANoe as in the ECU. For example:



The following mapping exists on ECU side:

Data Mapping							
4 of 4 data elements / operations are mapped. Use the Data Mapping Assistant to map the data elements / operations to signals.							
Mapping	Port Prototype	Data Element / Operation	Direction	Channel	PDU	Signal	
☒	PpTxCtrl	DesT0189D_A_b00r01	Tx	Flexray_1	p00T0189D_A_b00r01	sT0189D_A_b00r01	
☒	PpTxData	DeComFrSig_8Bit_5	Tx	Flexray_1	ComFr_3_5	ComFrSig_8Bit_5	
☒	PpRxCtrl	Desig02Rx_0x770_IL	Rx	CHNL	msgRx_0x770_IL_oCAN00	sig02Rx_0x770_IL	
☒	PpRxData	Desig03Rx_0x229_IL	Rx	CHNL	msgRx_0x229_IL_oCAN00	sig03Rx_0x229_IL	

The CAPL send function needs to send its Rx Ctrl value on the exact same signal on the exact same bus:

```
variables
{
    message CAN_1::msgRx_0x770_IL_oCAN00 rxCtrlMsg;
}
void writeRxCtrlSig(int data)
{
    rxCtrlMsg.sig02Rx_0x770_IL_omsgRx_0x770_IL_oCAN00 = data;
    output(rxCtrlMsg);
}
on sysvar StartApplication::RxCtrl
{
    writeRxCtrlSig(sysGetVariableInt(this));
}
```

For receiving values sent by the ECU, CAPL needs to listen to those bus-specific elements (like COM Signals), which are mapped and sent by the ECU:

Signal received → Signal handler changes system variable → Panel control is updated

```
on signal_update
FlexRay::p00T0189D_A_b00r01::sT0189D_A_b00r01_op00T0189D_A_b00r01
{
    onTxCtrl(this.raw);
}
void onTxCtrl(int data)
{
    sysSetVariableInt(sysvar::StartApplication::TxCtrl, data);
}
```

Diagnostic

Requesting diagnostic services and analyzing their results works in a similar manner. Instead of sending signals any of the connections configured in the BSW can be used, e.g. CanTp.

3.3 Bus Specific Mapping in this Delivery

Delivery specific mapping In this delivery, the following bus specific mapping was configured:

Variable	Bus specific element	Element type
Rx Ctrl	Signal_StartAppl_RxCtrl_MyECU_omsg_StartAppl_Rx_MyECU_oCAN	ComSignal
Rx Data	Signal_StartAppl_RxData_MyECU_omsg_StartAppl_Rx_MyECU_oCAN	ComSignal
Tx Ctrl	Signal_StartAppl_TxCtrl_MyECU_omsg_StartAppl_Tx_MyECU_oCAN	ComSignal
Tx Data	Signal_StartAppl_TxData_MyECU_omsg_StartAppl_Tx_MyECU_oCAN	ComSignal

In case the keyword “Generic Adapter” is used in the table above, a special handling for the control communication is implemented. More details can be found within the CanoePanelAbstraction.cin file.

In case the element type “SoAd Upper Layer PDU” is used in the table above, the control communication is handled via Ethernet. Therefore, a CDD is used which serves as Upper Layer of the SoAd module.

In case the bus specific elements contain the keyword “StartApplication” in the table above, they were added to the System Extract for test purposes. Please note that those bus specific elements will be removed next time an update of the configuration input files is performed.



Note: For variant configurations, the table contains the bus specific mapping of the default (main) variant. For more details see chapter 2.19.

4 Additional Information

4.1 Related Files

The following files are related to the Start Application:

	Directory	Description
StartApplication_sw_c .arxml	Config\ServiceComponents\	This file is the component type description of the Start Application SWC. It contains all ports, data elements, runnables etc.
StartApplication.c	Appl\Source\	This file contains the implementation of the instantiated component type SWC.
StartApplicationEth.c (only with ETH and/or control communication via Ethernet)	Appl\Source\	This file contains the implementation of the instantiated Start Application CDD which allows the communication with the Start Application over Ethernet (UDP). It is used for ETH use case and/or control communication over Ethernet.
StartApplicationScc.c (only with SCC)	Appl\Source\	This file contains the implementation of the SCC callback interface and the handling of the state machine.
StartApplicationSlave Core.c (only with Multicore)	Appl\Source\	This file contains the implementation of the instantiated component type SWC of slave cores in a Multicore use case.
StartApplicationMem oryAccessState.c (only with MemoryProtection)	Appl\Source\	This file contains the implementation of the Start Application Memory Access State CDD for the storage of the memory access violation detection state.
StartApplicationTrust ed.c (only with MemoryProtection)	Appl\Source\	This file contains the implementation of the instantiated component type SWC for the trusted part of the Start Application in a MemoryProtection use case.
StartApplicationIdenti tyManagerInformatio n.c (only with IDM)	Appl\Source\	This file contains the implementation of the Start Application Identity Manager Information CDD for the storage of the active variant in an IDM configuration.
BswInit.c	Appl\Source\	This file contains configuration dependent start up parts and task implementations.
Makefile.StartApplica tion	Appl\	This file contains references to the source files StartApplication.c and BswInit.c and is included in the project dependent makefile part.
StartApplication.cfg	Canoe\StartApplication\	CANoe configuration containing all panels to interact with the Start Application within the ECU.
StartApplicationSecO CProfile.xml	Canoe\StartApplication\ Configuration\	The security profile used for SecOC.

StartApplicationTest.t se	Canoe\StartApplication\	Test configuration referenced by the CANoe configuration.
PanelVars.vsysvar	Appl\Source\Capl\	System variables for interaction between Tester, ECU and Panels.
StartApplicationTest.xml	Appl\Source\Capl\	Test script for automated verification of the use cases.
CanoePanelAbstraction.can	Appl\Source\Capl\	Connects CANoe panels and bus specific send/receive handlers.
CanoePanelAbstraction.cin (only with Generic Adapter for control communication)	Appl\Source\Capl\	Generic Adapter specific extension of CanoePanelAbstraction.can.
StartApplicationSecOCMgr.can (only with SecOC)	Appl\Source\Capl\	Handle SecOC specific part for transmission and reception of signals.
StartApplication.h (only without RTE)	Appl\Include\	Replaces the SWC header in a no RTE use case.
StartApplicationEth.h (only without RTE and ETH and/or control communication over Ethernet)	Appl\Include\	Replaces the Start Application Eth CDD header in a no RTE use case combined with ETH use case and/or control communication over Ethernet.
StartApplication_MemMap.h (only without RTE)	Appl\Include\	Includes the memory mapping in a no RTE use case.
StartApplicationEth_MemMap.h (only without RTE and ETH and/or control communication over Ethernet)	Appl\Include\	Includes the memory mapping in a no RTE use case combined with ETH use case and/or control communication over Ethernet.
StartApplication_Compiler_Cfg.h (only without RTE)	Appl\Include\	Includes the compiler abstraction in a no RTE use case.
StartApplicationEth_Compiler_Cfg.h (only without RTE and ETH and/or control communication over Ethernet)	Appl\Include\	Includes the compiler abstraction in a no RTE use case combined with ETH use case and/or control communication via Ethernet.
StartApplicationScch.h (only with SCC)	Appl\Include\	This file contains the Start Application SCC API and callback declarations.
StartApplicationSlaveCore_MemMap.h (only with Multicore)	Appl\Include\	Includes the memory mapping in a Multicore use case.

StartApplicationMemoryAccessState.h (only with MemoryProtection)	Appl\Include\	This file contains the StartApplicationTrusted related API in a MemoryProtection use case.
StartApplicationIdentityManagerInformation.h (only with IDM)	Appl\Include\	Contains the IDM related API to set the active variant in an IDM configuration.
BswInit.h	Appl\Include\	Contains function declarations for BswInit.c.

4.2 VTT Dual Target

In case the Start Application was set up as a Dual Target project you can execute the Start Application either on hardware or in a virtual environment (VTT).

VTT

To use the VTT simulation, make the following changes in the CANoe configuration:

- > Set the Working Mode to Simulated bus.
- > Activate the node containing the DLL of the simulated ECU in the simulation setup.

Real Target

To use the real hardware, make the following changes in the CANoe configuration:

- > Set the Working Mode to Real bus.
- > Deactivate the node containing the DLL of the simulated ECU in the simulation setup.



Note: In case the XCP use case is part of the Start Application the A2L address update must be executed each time after switching between VTT and the real hardware.

4.3 Code Template Generation

All generated templates from Vector code generators work with special comments, which make it possible to add your implementation directly within the generated templates. All the code that is added between these special comment blocks won't be erased or overwritten if you regenerate the templates.

Vector code
templates

```
FUNC(void, Bac4_ApplMain_CODE) Dcm_BAC4_Jump2Boot(void) /* PRQA S 0850 */ /* MD_MSR_19.8 */
{
/*****
 * DO NOT CHANGE THIS COMMENT!          << Start of runnable implementation >>          DO NOT CHANGE THIS COMMENT!
 * Symbol: Dcm_BAC4_Jump2Boot
 *****/

    Srv_BootModeSet(SRV_BOOT_MODE_BOOTLOADER, (%Bac4_SampleAppl_ProgConditions));

/*****
 * DO NOT CHANGE THIS COMMENT!          << End of runnable implementation >>          DO NOT CHANGE THIS COMMENT!
 *****/
}
```

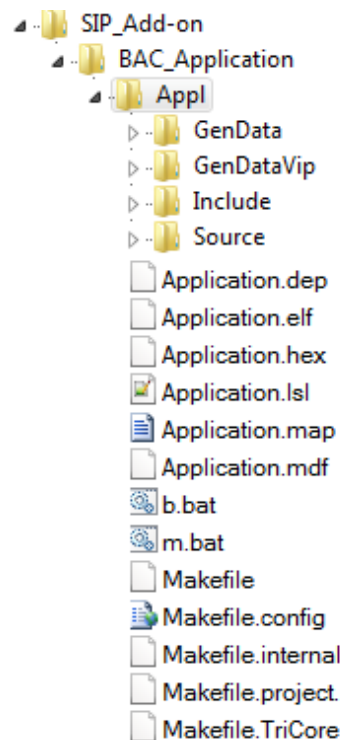


Caution: For more information about generated user templates please refer to the technical documentation of the RTE ([TechnicalReference_Asr_Rte.pdf](#)).

4.4 Adapt MakeSupport



Caution: Using the build environment (MakeSupport) that is part of the framework application is optional. Please note that Vector does not take over any liability for the delivered build environment.



If no other build environment is available, the MakeSupport environment may be used to build the framework application or to set up an evaluation project.

Depending on your project you may have to adapt some of the MakeSupport files. This chapter shall help you with this.

The MakeSupport is split into two parts: A generic part which can be used without project specific adaptations and a project specific part which has to be adapted.

The MakeSupport uses several configuration files located in the Appl folder of the project. Please have a look at the following list to find out which files may have to be modified and which file must not be edited!



Note: The following files have to be edited to fit your project needs.

Directory / File	Description
Appl\Makefile.config	Compiler and linker settings.
Appl\Makefile	Compiler and linker options and settings such as the path to the compiler.
Appl\Makefile.Project.Part.defines	Specifies which BSW Modules and which application specific .C files shall be included into the build process.
Appl\m.bat	Batch file to access the MakeSupport functionality. If the folder structure is changed, the path to the MakeSupport directory has to be adapted here.



Do not edit the following files manually:

Directory / Files	Description
SIP Add-on\MakeSupport	Project independent part of the MakeSupport. Consists of generic parts like target and dependency generation.
Appl\Makefile.<Controller>.<Compiler>.<Emulator>.make	Hardware-specific makefile
Appl\b.bat	Batch file to trigger a re-build of the project. Uses m.bat.

4.4.1 Useful Commands

In order to use the MakeSupport the following command line options might be helpful. Please open a command line window to the Appl folder of the framework application and run one of the following commands.

Command	Description
m	Builds the project using the configured compiler (only changes will be considered)
b	Performs a rebuild of the project (same effect as "m rebuild")
m depend	Creates dependencies (i.e. a preprocessor run collects all required header files from the configured header file locations).
m help	Shows all possible commands. Please note, that some commands require further tools which are not part of this delivery.

4.4.2 Change Compiler Settings

The MakeSupport requires some information on the compiler environment in order to work. Most likely the compiler directory has to be adapted. Depending on the type of compiler, other options, such as the compiler or linker version, the host name of the compiler license server etc. have to be configured. Please observe the file `Makefile` for other settings required by the compiler that is used by your project.

Compiler Directory

Make files variables	Steps to change it
COMPILER_BASE	1. Open <code>Makefile</code> 2. Search for <code>COMPILER_BASE</code> and enter the path, where your compiler is located. 3. Save <code>Makefile</code>



Note: Some versions of the MakeSupport might not provide a `COMPILER_BASE` setting in the file `Makefile`. In this case the setting might have to be done in the file `Makefile.config`.

4.4.3 Add and Delete *.c and *.h file (e.g. AR Software Components)

This option allows adding or removing *.c files (e.g. AUTOSAR Software Components) to the MakeSupport environment.

Make files variable	Steps to change it
APP_SOURCE_LIST ADDITIONAL_INCLUDES	1. Open <code>Makefile.project.part.defines</code> 2. Search for <code>APP_SOURCE_LIST</code> and enter/delete relative path to the *.c file. 3. Search for <code>ADDITIONAL_INCLUDES</code> and enter/delete the path to the folder, where the *.h files are located. Please note, that this setting is required only if header files are provided in a non-standard location. 4. Save <code>Makefile.project.part.defines</code>

4.4.4 Change the Project Name

The project name determines the name of the created output files and may be adapted.

Make file variables	Steps to change it
PROJECT_NAME	1. Open <code>Makefile.config</code> 2. Search for <code>PROJECT_NAME</code> and enter new project name 3. Save <code>Makefile.config</code> 4. Build project

4.4.5 Deactivate MICROSAR BSW Modules

Typically, the delivered MakeSupport environment includes all ordered BSW modules to the project. If some modules shall be removed as the modules are not used, these modules have to be disabled.

1. Open `Makefile.project.part.defines`. The software modules are sorted in alphabetic manner.
2. Search for your modules and activate/deactivate software modules.
3. To activate enter 1, to deactivate enter 0. Please note that the listed BSW modules use the Vector internal names e.g. `GW_ASRPDUR` for `PDUR`.
4. Save `Makefile.project.part.defines`

4.4.6 Advanced MakeSupport Settings

The following settings are rarely used but may be of interest for some projects.

Add Assembler files

1. Create a folder and add new assembler file.
2. Open `Makefile`
3. Search for `ASM_SOURCES` and enter relative path to the new assembler file.
4. Save `Makefile`
5. Build project



Note: Make sure that your additional assembler file has the same suffix as defined in the Makefile (`ASM_SUFFIX`).

Add Object files

1. Create a folder and add new object file.
2. Open `Makefile`
3. Search for `ADDITIONAL_OBJECTS` and enter relative path to the new object file.
4. Save `Makefile`
5. Build project

5 Appendix A: Addresses

Addresses on Vector homepage

Please find the contacts of Vector Informatik GmbH and all subsidiaries worldwide via:

http://www.vector.com/vi_addresses_en.html



More Information

- > News
- > Products
- > Demo Software
- > Support
- > Training Classes
- > Addresses

www.vector.com